

This is a README.md

Java笔记:

2026年3月30日:

自动生成局部变量:

变量.`var` +回车

快速数组遍历:

数组名.`for` i +回车

Java数组元素反转---使用“双指针法”在原地 (in-place) 反转一个数组:

```
public static void reverseArray(int[] arr){
    // 定义了start开始和end结束 | 定义了循环条件，无论数组长度是偶或奇，都会结束 | 每循环一次，
    start+1,end-1
    for(int start = 0 , end = arr.length -1 ;start < end; start++ , end--){
        //典型的交换数组元素
        int temp = arr[start];
        arr[start] = arr[end];
        arr[end] = temp;
    }
}
```

Java水仙花数案例---练习For循环和If判断:

```
public static int getCount(){
    // 主要是对每一个数进行取模，然后拿出来乘3次方之后相加
    int count = 0;
    for (int i = 100 ; i <= 999 ; i++){
        int ge = i % 10;
        int shi = i / 10 % 10;
        int bai = i / 100;
        if (ge * ge * ge + shi * shi * shi + bai * bai * bai == i) {
            System.out.println(i);
            count++;
        }
    }
    return count;
}
```

2026年4月2日，因为昨天晚上没睡好，今天没有精神，特别瞌睡，给自己放假一天

2026年4月3日:

JavaBean：一套约定俗成的类写法。规范，1.公共无参构造器，2.属性私有化,3.提供Getter/Setter

JavaBean的主要应用场景：通常用于数据的传输

Constructor构造方法（构造器），在创建对象时自动执行构造器，且允许重载

封装：内部私有，外部接口.保护数据，隐藏复杂性

```

public class Account {
    private double balance; // 私有属性，防止被外部直接修改

    public void deposit(double amount) {
        if (amount > 0) {
            this.balance += amount; // 封装了业务逻辑
        }
    }
}

```

继承：孩子继承父亲的家产.实现代码复用，定义层级

```

// 1. 继承：建立父类
class Animal {
    String name;

    public Animal(String name) {
        this.name = name;
    }

    // 父类定义一个通用的方法
    public void makeSound() {
        System.out.println("动物在发出声音...");
    }
}

// 2. 继承：Dog 继承自 Animal
class Dog extends Animal {
    public Dog(String name) {
        super(name); // 调用父类的构造函数
    }

    // 重写 (Override)：这是实现多态的关键
    @Override
    public void makeSound() {
        System.out.println(name + " 说：汪汪!");
    }
}

// 2. 继承：Cat 继承自 Animal
class Cat extends Animal {
    public Cat(String name) {
        super(name);
    }

    @Override
    public void makeSound() {
        System.out.println(name + " 说：喵喵!");
    }
}

// 3. 多态的实际应用

```

```

public class Main {
    public static void main(String[] args) {
        // 多态的核心：父类引用指向子类对象
        // 虽然类型都是 Animal，但表现出的行为不同
        Animal myDog = new Dog("大黄");
        Animal myCat = new Cat("小花");

        // 调用相同的方法名，结果却不同
        performSound(myDog); // 输出：大黄 说：汪汪！
        performSound(myCat); // 输出：小花 说：喵喵！
    }

    // 这是一个多态方法：它只接受 Animal 类型
    // 这样无论以后增加什么动物，这个方法都不用改
    public static void performSound(Animal animal) {
        animal.makeSound();
    }
}

```

多态：一份蓝图，多种实现.提高灵活性，支持模块化扩展

```

public interface Payment {
    void pay(double amount);
}

public class AliPay implements Payment {
    public void pay(double amount) { System.out.println("支付宝支付: " + amount); }
}

public class WeChatPay implements Payment {
    public void pay(double amount) { System.out.println("微信支付: " + amount); }
}

// 架构师视角：业务代码只依赖接口，不依赖具体实现
public class OrderService {
    public void process(Payment method, double total) {
        method.pay(total); // 运行时根据传入的实例决定调用哪个方法
    }
}

```

过度封装会增加开发成本，过度继承会导致类爆炸，而没有多态则会让系统僵死。

2026年4月4日，今天需要把马原过一遍，因为离自考本还有一两周,所以今天休息一天

2026年4月5日：

this 和 super：

this：代表本类对象的引用。

super：代表父类存储空间的标识

Object 类：

所有的类，都直接或间接继承了 Object 类（祖宗类）

而 Object 中的 toString 就是让你重写的，因为在你创建类的时候自动继承 Object 类：

```
@Override
public String toString() {
    return "Student{" +
        "age=" + age +
        ", name='" + name + '\'' +
        '}';
}
```

final 关键字：是最终的意思，可以修饰（方法，类，变量）

基本数据类型变量：数据值不可改变

引用数据类型变量：地址值不可改变

final 修饰成员变量，初始化时机

1. 在定义的时候完成赋值
2. 在构造方法结束之前完成赋值。

```
class A {
    final int num;

    public A(int num) {
        this.num = num;
    }
}
```

抽象类：是一种特殊的父类，内部可以编写抽象方法

抽象方法 abstract：特点，强制要求子类重写

```
abstract class Animal {
    public abstract void eat();

    public void drink(){
        System.out.println("喝水");
    }
}
```

/*

抽象类的注意事项：

1. 抽象类不能实例化
原因：如果允许创建对象，就可以调用抽象方法了，这样做没意义
2. 抽象类不能创建对象，里面还是存在构造方法
原因：构造方法是交给子类通过 super() 调用的。
3. 抽象类中可以编写普通的方法
原因：子类可以直接继承到这些方法使用。
4. 抽象类的子类
A：重写所有抽象方法
B：将子类改造为抽象类

*/

2026年4月6日，今天因为要推比赛进度，一直在弄香橙派和pegasus，所以今天休息一天

2026年4月7日，复习自考本和弄pegasus

2026年4月8日，复习自考本

2026年4月9日，复习自考本

2026年4月10日，复习自考本

2026年4月11日，自考本考试

2026年4月12日，自考本考试

2026年4月13日，做比赛项目，因为项目15号截止

2026年4月14日，做比赛项目

2026年4月15日：

设计模式：一些优秀的程序员，总结出来的经验

模板设计模式：将抽象类整体看作为一个模板，模板中不能决定的内容，定义为抽象方法 让使用模板的类，补齐抽象方法的逻辑

`final`，可以限制子类不可重写

接口：体现的思想是对规则的声明，Java中的接口更多体现的是对行为的抽象

接口定义和使用：接口关键字 `interface` 来定义，接口和类之间是实现关系，通过 `implements` 关键字表示

2026年4月16日：

IDEA快捷键：Ctrl+R 一键替换

2026年4月19日：

栈：先进后出

队列：先进先出

数据结构：

数组：查询与更改快，增删慢

链表：查询与修改慢，增删相对于数组较快，首尾操作极快

泛型：1.统一数据类型。2.将运行期的错误提升到了编译期。（如果没有指定类型默认为Object）

泛型常见标识符：E:Element元素。T:Type类型。K:Key键。V:Value值

泛型非静态方法：泛型是根据类的泛型去匹配的

泛型静态方法：需要声明出自己独立的泛型

2026年4月20日：

TreaSet:

- TreeSet保留重复的方法：CompareTo比较过后，返回非0数字(1或-1最好)进行保留
- TreeSet中比较器排序优先于自然排序（重写CompareTo方法的排序）
- TreeSet比较器排序，参数1-参数2，正序。参数2-参数1倒序

HashSet:

- HashSet集合保证元素唯一，需要同时重写hashCode和equals方法
- HashSet底层结构 (jdk8版本)：数组+链表+红黑树
- HashSet底层原理：(jdk8以前头插法，jdk8以后尾插法) 字符串比较时计算它的取模值(哈希值)，例如数组长度15，取模值15。JDK8开始，当链表长度超过8，且数组长度 ≥ 64 时，自动将链表转换为红黑树
- HashSet当数组存满到 $16 * 0.75 = 12$ 时，就自动扩容，每次扩容为原先容量的两倍 (0.75为加载因子)

LinkHashSet:有序，不重复

应用场景：想要集合中的元素可重复，就用ArrayList集合。相对集合中的元素去重，用HashSet,基于哈希表

相对集合中的元素去重，而且保证存取顺序，用LinkedHashSet集合，基于哈希表和双链表，效率低于HashSet

相对集合中的元素进行排序，用TreeSet集合，基于红黑树

Collections工具类：可变参数，可以在形参中接收多个数据，格式 数据类型...参数名称

Map集合：一种双列集合，每个元素包含两个数据，元素格式 key = value (键值对格式)

Map常用的API：map.remove 删除(根据键名删除键值对), map.put 添加或覆盖，如果键名相同，则覆盖前一个的value值，并返回

map.containsKey, 按键查找。 map.containsValue, 按值查找

Map遍历的三种方式：

1. 键找值 (通过 keySet 获取键，通过 get 获取值)

```
public static void main(String[] args) {
    Map<String, String> map = new HashMap<>();
    map.put("张三", "北京");
    map.put("李四", "上海");
    map.put("王五", "成都");

    // 1. 获取Map集合中，所有的键
    Set<String> keySet = map.keySet();
    // 2. 遍历set集合，获取每一个键
    for (String key : keySet) {
        // 3. 通过map集合的get方法，根据键查找对应的值
        String value = map.get(key);
        System.out.println(key + "---" + value);
    }
}
```

2. 通过键值对对象获取键和值

```
public static void main(String[] args) {
    Map<String, String> map = new LinkedHashMap<>();
    map.put("张三", "北京");
    map.put("李四", "北京");
    map.put("王五", "北京");

    // 1. 调用entrySet方法，获取所有的键值对对象。
```

```

        Set<Map.Entry<String, String>> entrySet = map.entrySet();
        // 2. 遍历Set集合，获取每一个键值对对象
        for (Map.Entry<String, String> entry : entrySet) {
            // 3. 通过键值对对象，获取键和值
            String key = entry.getKey();
            String value = entry.getValue();
            System.out.println(key + "---" + value);
        }
    }
}

```

3. 通过foreach方法遍历

```

public static void main(String[] args) {
    Map<String, String> map = new TreeMap<>();
    map.put("张三", "北京");
    map.put("李四", "北京");
    map.put("王五", "北京");

    map.forEach((key, value) -> System.out.println(key + "---" + value));
    map.forEach((k, v) -> System.out.println(k + "---" + v));
}

```

Map接口：

- 双列集合的数据结构，都只针对于键有效，和值没有关系
- TreeMap：键(红黑树)(键排序)
- HashMap：键(哈希表)(键唯一)
- LinkedHashMap：键(哈希表 + 双向链表)(键唯一，并保证顺序)

Stream流：配合Lambda表达式，简化集合和数组操作

- (IDEA快捷键)Ctrl + Alt + M 代码段直接变成方法(选中代码段)
- 流中的操作不会修改数据源,需要做Stream流的收集操作

2026年4月22日：异常

Exception异常：

- 运行时异常
- 编译时异常(如日期解析异常)

解决方式：

- 抛出异常(throws)
- 捕获异常(try...catch)
- Ctrl + T 快捷键选择常用的逻辑代码
- `e.printStackTrace` 和 `throw new xxx`

运行异常代码部分：（运行异常继承RuntimeException）

```

package day15_Exception;

```

```

public class ExceptionDemo1 {
    // 主代码
    public static void main(String[] args) {
        int x = 101;
        System.out.println("开始了");
        try {
            save(x);
            System.out.println("执行成功了");
        } catch (Exception e) {
            e.printStackTrace(); // 打印异常对象信息
            System.out.println("执行失败了");
        }
        System.out.println("结束了");
    }

    private static void save(int age) {

        if (age <= 0 || age > 100) {
            throw new ArithmeticException("/x is error");
        }
        System.out.println("年龄保存成功了，年龄是: "+age);

    }

}

```

```

package day15_Exception;

public class ARuntimeException extends RuntimeException {
    //自定义的运行异常代码
    public ARuntimeException(String message) {
        super(message);
    }
}

```

开发中对于异常的常见处理方式：

- 捕获异常，记录异常并响应合适的信息给用户
- 捕获异常，尝试重新修复

File类：

- 绝对路径: `File f = new File("D:\\A.txt")` ,没有文件或文件夹返回false
- 相对路径:相对于项目
- `f.getAbsolutePath()` 获取绝对路径
- `File f = new File("A.txt"), f.createNewFile()` 创建文件

File类常见的API (判断相关)：

- `f.isDirectory()` ,判断此路径名表示的File是否为文件夹

- `f.isFile()` ,判断此路径名表示的File是否为文件
- `f.exists()` ,判断此路径名表示的File是否存在

File类常见的API (获取相关) :

- `f.length()` ,返回文件的大小(字节数量)
- `f.getAbsolutePath()` ,返回文件的绝对路径
- `f.getPath()` ,返回定义文件时使用的路径
- `f.getName()` ,返回文件的名称,带后缀
- `f.lastModified()` ,返回文件的最后修改时间(时间毫秒值)

File类常见的API (创建和删除相关) :

- `f.createNewFile()` ,创建一个新的空的文件
- `f.mkdir()` ,只能创建一级文件夹
- `f.mkdirs()` ,可以创建多级文件夹
- `f.delete()` ,删除文件或文件夹(强制删除)

File类常见的API (遍历相关) :

- `File[] listFiles()` ,获取当前目录下所有的"一级文件对象",返回File数组

```
import java.io.File;

public class FileTest2 {
    /*
        需求：键盘录入一个文件夹路径，找出这个文件夹下所有的 .java 文件

        public File[] listFiles()  获取当前目录下所有的 “一级文件对象” 返回 File 数组
    */
    public static void main(String[] args) {

        File dir = FileTest1.getDir();

        printJavaFile(dir);
    }

    private static void printJavaFile(File dir) {
        // 1. 获取当前文件夹下所有的文件和文件夹对象
        File[] files = dir.listFiles();
        // 2. 遍历数组，获取每一个文件和文件夹对象
        for (File file : files) {
            // 3. 判断当前对象是否是文件，并且是java文件
            if(file.isFile() && file.getName().endsWith(".java")){
                // 4. 打印在控制台
                System.out.println(file);
            }
        }
    }
}
```

递归:方法直接或间接调用本身

递归如果没有控制好终止，会出现递归死循环，导致内存溢出现象

```
public class RecursionDemo2 {
    /*
        需求：使用递归求5的阶乘

        5的阶乘（5!）： 5 * 4 * 3 * 2 * 1

        5的阶乘（5!）： 5 * 4!
        4的阶乘（4!）： 4 * 3!
        3的阶乘（3!）： 3 * 2!
        2的阶乘（2!）： 2 * 1!
        1的阶乘（1!）： 1

    */
    public static void main(String[] args) {
        System.out.println(jc(5));
    }

    public static int jc(int num) {
        if (num == 1) {
            return 1;
        } else {
            // 思路：需要调用一个方法，获取4的阶乘
            // 5 * jc(4)
            // 4 * jc(3)
            // ...
            return num * jc(num - 1);
        }
    }
}
```

递归练习——不死身兔（斐波那契数列）：

```
public class RecursionTest {
    /*
        有一对兔子，从出生后第3个月起每个月都生一对兔子
        小兔子长到第三个月后每个月又生一对兔子，假如兔子都不死，问第二十个月的兔子对数为多少？

        规律：从第三个月开始，兔子的对数是前两个月相加的和
    */
    public static void main(String[] args) {
        System.out.println(get(20));
    }

    public static int get(int month) {
        // 第一个，第二个月兔子的对数为1
        if (month == 1 || month == 2) {
            return 1;
        } else {
            // 第三个月：第一个月 + 第二个月    month 3
            // 第四个月：第二个月 + 第三个月    month 4
        }
    }
}
```

```

        // ...
        return get(month - 2) + get(month - 1);
    }
}
}

```

递归练习——找出所有.java文件

```

public class FileTest3 {
    /*
        需求：键盘录入一个文件夹路径，找出这个文件夹下所有的 .java 文件（考虑子文件夹）

    */
    public static void main(String[] args) {
        File dir = FileTest1.getDir();
        printJavaFile(dir);
    }

    private static void printJavaFile(File dir) {
        // 获取当前目录下，所有文件和文件夹对象
        File[] files = dir.listFiles();
        // 遍历数组，获取每一个文件和文件夹对象
        for (File file : files) {
            // 如果是文件，并且是.java文件，就打印在控制台
            if (file.isFile() && file.getName().endsWith(".java")) {
                System.out.println(file);
            } else if (file.isDirectory()) {
                // 说明是文件夹，进入这个文件夹，继续查找.java文件
                // 思路：需要调用一个方法，进入文件夹查找.java文件，发现自己这个方法就是解决此问题，递归
                调用。

                if (file.listFiles() != null) {
                    printJavaFile(file);
                }
            }
        }
    }
}

```

递归练习——删除文件夹

```

public class FileTest4 {
    /*
        需求：设计一个方法，删除文件夹
        注意：delete() 只能删除空文件夹

    */
    public static void main(String[] args) {
        File dir = new File("E:\\test");
        deleteDir(dir);
    }

    private static void deleteDir(File dir) {
        File[] files = dir.listFiles();
    }
}

```

```

        for (File file : files) {
            if (file.isFile()) {
                file.delete();
            } else {
                if (file.listFiles() != null) {
                    deleteDir(file);
                }
            }
        }
        dir.delete();
    }
}

```

递归练习——统计文件夹大小

```

public class FileTest5 {
    /*
        需求：键盘录入一个文件夹路径，统计文件夹的大小
    */
    public static void main(String[] args) {
        File dir = FileTest1.getDir();

        long length = getDirLength(dir);

        System.out.println("字节数量为:" + length);
    }

    private static long getDirLength(File dir) {
        long result = 0;
        File[] files = dir.listFiles();
        for (File file : files) {
            if (file.isFile()) {
                result += file.length();
            } else {
                if (file.listFiles() != null) {
                    result += getDirLength(file);
                }
            }
        }
        return result;
    }
}

```

2026年4月22日：常用API与多线程

Math类：

```

/*
Math类 : 包含执行基本数字运算的方法

-----

public static int abs (int a) : 获取参数绝对值
public static double ceil (double a) : 向上取整
public static double floor (double a) : 向下取整
public static int round (float a) : 四舍五入
public static int max (int a, int b) : 获取两个int值中的较大值
public static double pow (double a,double b) : 返回a的b次幂的值
public static double random () : 返回值为double的随机值, 范围[0.0,1.0)
-----

*/

```

System类:

```

/*
System类常见方法 :

1. public static void exit (int status) : 终止当前运行的 Java 虚拟机, 非零表示异常终止
2. public static long currentTimeMillis () : 返回当前系统的时间毫秒值形式
    - 返回1970年1月1日 0时0分0秒, 到现在所经历过的毫秒值
    - 返回1970年1月1日 8时0分0秒, 到现在所经历过的毫秒值

3. public static void arraycopy(Object src, int srcPos, Object dest, int destPos,
int length) : 数组拷贝

    1. 数据源数组
    2. 起始索引
    3. 目的地数组
    4. 目的数组的起始索引
    5. 拷贝的个数

    System.arraycopy([源数组],[起始索引],[目的数组],[目的数组起始索引],[拷贝的个数])

*/

```

包装类: 将基本数据类型, 包装成类(变成引用数据类型)

除了 `Integer(int)`, `Character(char)` 两种其余都是首字母大写

基本数据类型包装成类:

```

/*
将基本数据类型, 包装成类 (变成引用数据类型)

byte          Byte
short         Short
int           Integer
long          Long
char          Character
float         Float

```

double	Double
boolean	Boolean

手动装箱：手动调用 `Integer.valueOf()` 方法，将基本数据类型，手动包装为类

手动拆箱：手动调用 `Integer`中的 `intValue()` 方法，将包装类对象，转换为基本数据类型

自动拆装箱：基本数据类型和对应的包装类，可以直接运算，操作起来非常便捷

*/

Integer类的常见方法:

/*

`Integer`类的常见方法:

<code>public static String toBinaryString(int i)</code>	得到二进制
<code>public static String toOctalString(int i)</code>	得到八进制
<code>public static String toHexString(int i)</code>	得到十六进制
<code>public static int parseInt(String s)</code>	将字符串类型的整数转成 <code>int</code> 类型的整数

*/

练习:

/*

已知字符串

`String s = "10,50,30,20,40";`

请将该字符串转换为整数并存入数组

随后求出最大值打印在控制台

*/

```
public static void main(String[] args) {
    String content = "10,50,30,20,40";

    String[] arr = content.split(",");

    int[] nums = new int[arr.length];

    for (int i = 0; i < arr.length; i++) {
        nums[i] = Integer.parseInt(arr[i]);
    }

    int max = nums[0];
    for (int i = 1; i < nums.length; i++) {
        if (nums[i] > max) {
            max = nums[i];
        }
    }

    System.out.println("最大值为:" + max);
}
```

BigDecimal类: (解决小数运算精度损失问题)

```

/*
BigDecimal类：用于解决小数运算中，出现的不精确问题

BigDecimal创建对象：

    public BigDecimal(double val)：不推荐，无法保证小数运算的精确
    -----
    public BigDecimal(String val)
    public static BigDecimal valueOf(double val)

BigDecimal常用成员方法：

    public BigDecimal add(BigDecimal b)：加法
    public BigDecimal subtract(BigDecimal b)：减法
    public BigDecimal multiply(BigDecimal b)：乘法
    public BigDecimal divide(BigDecimal b)：除法
    public BigDecimal divide(另一个BigDecimal对象, 精确几位, 舍入模式)：除法

    RoundingMode.HALF_UP：四舍五入
    RoundingMode.UP：进一法
    RoundingMode.DOWN：去尾法

注意：如果使用BigDecimal运算，出现了除不尽的情况，就会出现异常
*/

```

Arrays数组工具类:

```

/*
Arrays 数组操作工具类，专门用于操作数组元素

    public static String toString(类型[] a)                将数组元素拼接为带有格式的字符串
    public static boolean equals(类型[] a, 类型[] b)        比较两个数组内容是否相同（元素，个数，顺序）
    public static int binarySearch(int[] a, int key)        查找元素在数组中的索引（二分查找法）
                                                            - 注意：操作的数组，必须是排好顺序。
    public static void sort(类型[] a)                      对数组进行默认升序排序
*/

```

Data类:

```

/*
Date类：表示时间的类

1. 构造方法：

    public Date()：将当前时间，封装为Date日期对象
    public Date(long time)：把时间毫秒值转换成Date日期对象

2. 常见方法：

```

```
public long getTime() : 返回从1970年1月1日 00:00:00走到此刻的总的毫秒数
public void setTime(long time) : 设置日期对象的时间为当前时间毫秒值对应的时间
```

```
*/
```

SimpleDateFormat类:(时间格式化)

```
/*
```

```
SimpleDateFormat类 : 用于日期格式化
```

1. 构造方法 :

```
public SimpleDateFormat() : 创建一个日期格式化对象, 使用 [默认模式]
```

```
public SimpleDateFormat(String pattern) : 创建一个日期格式化对象, [手动指定模式]
```

2. 常用方法 :

```
public final String format(Date date) : 将日期对象, 转换为字符串
```

```
public final Date parse(String source) : 将日期字符串, 解析为日期对象
```

```
*/
```

LocalDateTime类:

```
/*
```

```
-----
LocalDate、LocalTime、LocalDateTime
```

对象的创建方式:

1. now() : 当前时间

2. of(...) : 设置时间

```
-----
LocalDateTime 转换LocalDate, LocalTime
```

1. toLocalDate()

2. toLocalTime()

```
*/
```

修改时间:

```
public static void main(String[] args) {
    LocalDateTime nowTime = LocalDateTime.now();

    // 当前时间
    System.out.println(nowTime);
    // minus : 减去
```



```

        // minusYears(年), minusMonths(月), minusDays(日), minusWeeks(周), minusHours(时),
        minusMinutes(分), minusSeconds(秒), minusNanos(纳秒)
        System.out.println("减一小时:" + nowTime.minusHours(1));
        System.out.println("减一分钟:" + nowTime.minusMinutes(1));
        System.out.println("减一秒钟:" + nowTime.minusSeconds(1));
        System.out.println("减一纳秒:" + nowTime.minusNanos(1));

        System.out.println("对比时间, 确定方法返回的都是新的实例 >>>>>> " + nowTime);

        System.out.println("-----");

        // plus : 加
        // plusYears(年), plusMonths(月), plusDays(日), plusWeeks(周), plusHours(时),
        plusMinutes(分), plusSeconds(秒), plusNanos(纳秒)
        System.out.println("加一小时:" + nowTime.plusHours(1));
        System.out.println("加一分钟:" + nowTime.plusMinutes(1));
        System.out.println("加一秒钟:" + nowTime.plusSeconds(1));
        System.out.println("加一纳秒:" + nowTime.plusNanos(1));

        System.out.println("-----");

        // with : 这里体现出的是, 设置效果
        System.out.println("修改的效果:");
        // withYear(年), withMonth(月), withDayOfMonth(日), withHour(时), withMinute(分),
        withSecond(秒), withNano(纳秒)
        System.out.println(nowTime.withYear(2008));
        System.out.println(nowTime.withMonth(8));
        System.out.println(nowTime.withDayOfMonth(8));
        System.out.println(nowTime.withHour(8));
        System.out.println(nowTime.withMinute(8));
        System.out.println(nowTime.withSecond(8));
        System.out.println(nowTime.withNano(8));
        System.out.println("-----");

        LocalDate myDate = LocalDate.of(2008, 8, 8);
        LocalDate nowDate = LocalDate.now();

        // 2008-08-08是否在nowDate之前?
        System.out.println(myDate + "是否在" + nowDate + "之前? " +
            myDate.isBefore(nowDate));

        // 2008-08-08是否在nowDate之后?
        System.out.println(myDate + "是否在" + nowDate + "之后? " + myDate.isAfter(nowDate));
        System.out.println("-----");

        // 判断两个时间是否相同
        System.out.println(myDate.equals(nowDate));
    }

```

DateTimeFormatter类:(格式化时间):

```
/*
```

用于时间的格式化和解析:

1. 对象的获取：

`static DateTimeFormatter ofPattern(格式)：` 获取格式对象

2. 格式化：

`String format(时间对象)：` 按照指定方式格式化

3. 解析：

`LocalDateTime.parse("解析字符串", 格式化对象)；`

`LocalDate.parse("解析字符串", 格式化对象)；`

`LocalTime.parse("解析字符串", 格式化对象)；`

`*/`

ChronoUnitDemo:

```
/**
 * ChronoUnit可用于在单个时间单位内测量一段时间，这个工具类是最全的了，可以用于比较所有的时间单位
 */
```

IO流体系结构：字节流(万能流)和字符流(纯文本文件)

字节流操作中文可能有乱码,这个时候就可以用字符流来代替

字节流:

FileOutputStream(字节输出流):

```
/*
    字节流写出数据

    构造方法:
        1. public FileOutputStream(String name)：输出流关联文件，文件路径以字符串形式给出
        public FileOutputStream(String name, boolean append)：参数2为追加写入的开关
        2. public FileOutputStream(File file)：输出流关联文件，文件路径以File对象形式给出
        public FileOutputStream(File file, boolean append)：参数2为追加写入的开关

    成员方法:
        public void write(int i)：写出一个字节
        public void write(byte[] b)：写出一个字节数组
        public void write(byte[] b, int off, int len)：写出字节数组的一部分

    细节:
        输出流关联文件，文件如果不存在：会自动创建出来
        如果文件存在：会清空现有的内容，然后再进行写入操作
*/
```

记得 `.close()` 关闭流,不然资源无法释放

FileOutputStream异常代码处理:

```
/*
    IO流的异常处理方式: jdk7版本开始

    try
        (需要调用close方法的流对象)
        {
            IO流逻辑代码
        }
    catch (异常类名 对象名) {
        异常处理方案
    }

    ()中的流对象, 会自动调用close方法, 就算有异常, 也会调用close.
*/
```

FileInputStream(输入流):

```
/*
    FileInputStream 字节输入流读取数据

    1. 构造方法:

        FileInputStream(String name)    输入流关联文件, 文件路径以字符串形式给出
        FileInputStream(File file)      输入流关联文件, 文件路径以File对象形式给出

    2. 成员方法:

        public int read()                读取单个字节
        public int read(byte[] bys)      读取一个字节数组
                                           - 将读取到的字节, 存入数组容器, 返回读取到的有效字
                                           节个数
*/
public static void main(String[] args) throws IOException {
    FileInputStream fis = new FileInputStream("E:\\A.txt");

    byte[] bys = new byte[2];

    // 将读取到的字节, 存入数组容器, 返回读取到的有效字节个数
    int len;
    while( (len = fis.read(bys)) != -1) {
        String s = new String(bys, 0, len);
        System.out.print(s);
    }

    fis.close();
}
```

```
private static void method1() throws IOException {
    FileInputStream fis = new FileInputStream("E:\\A.txt");

    int i;
    while( (i = fis.read()) != -1 ){
        System.out.print((char)i);
    }

    fis.close();
}
```

如果读取不到(也就是读完了)会返回 -1

字符流:

FileReader(字符流读取):

```
/*
FileReader: 用于读取纯文本文件, 解决中文乱码问题

构造方法:
1. public FileReader(String fileName) 字符输入流关联文件, 路径以字符串形式给出
2. public FileReader(File file) 字符输入流关联文件, 路径以File对象形式给出

成员方法:
public int read() : 读取单个字符
public int read(char[] cbuf) : 读取一个字符数组, 返回读取到的有效字符个数
*/
```

FileWriter(字符流输出):

```
/*
FileWriter字符输出流写出数据 :

构造方法:
字符串形式给出
FileWriter(String fileName): 字符输出流关联文件, 路径以
File对象形式给出
FileWriter(String fileName, boolean append): 参数2: 追加写入的开关
FileWriter(File file): 字符输出流关联文件, 路径以
FileWriter(File file, boolean append): 参数2: 追加写入的开关

成员方法:
public void write(int c) 写出单个字符
public void write(char[] cbuf) 写出一个字符数组
public void write(char[] cbuf, int off, int len) 写出字符数组的一部分
public void write(String str) 写出字符串
public void write(String str, int off, int len) 写出字符串的一部分
*/
```

FileWriter需要刷出数据(类似于一个缓冲区, 需要刷一下):

```
/*
```

注意事项：字符输出流写出数据，需要调用flush或close方法，数据才会写出

flush() ：刷出数据，刷出后可以继续写出

close() ：关闭流释放资源，顺便刷出数据，关闭后不可以继续写出

```
*/
```

字节流使用场景:不是纯文本文件都用字节流

字符流使用场景:读写纯文本文件

Properties集合和方法:

集合:

```
/*
```

Properties作为集合的使用

Object setProperty (String key, String value) : 类似Map集合的put方法

String getProperty (String key) : 类似Map集合的get方法

Set<String> stringPropertyNames() : 类似Map是集合的keySet方法

```
*/
```

方法 (类) :

```
/*
```

Properties 和 IO 有关的方法

void load(InputStream inStream) 从输入字节流读取属性列表（键和元素对）

void load(Reader reader) 从输入字符流读取属性列表（键和元素对）

void store(OutputStream out, String comments) 将集合的键值对写出到文件(字节流)

void store(Writer writer, String comments) 将集合的键值对写出到文件(字符流)

```
*/
```

Properties 最常见用途就是:存配置

比如:数据库连接信息、项目环境配置、系统参数

```
username=root
```

```
password=123456
```

```
url=jdbc:mysql://localhost:3306/test
```

Hutool工具(提升开发效率):

帮你把Java开发中常见的"重复、繁琐操作"封装好，让你少写很多代码

.jar 文件就是一个压缩包,里面压缩的字节码文件,也就是.class 文件

```

/*
Ioutil常用方法
    copy(InputStream in, OutputStream out, int bufferSize)    字节流拷贝
    copy(Reader reader, Writer writer)                        字符流拷贝
    readLines(Reader reader, Collection<String> collection)    按行读取内容到集合
    close(Closeable closeables)                               安全关闭流

FileUtil常用方法
    touch(filePath)                                            创建文件（自动创建父目
录）
    mkdir(dirPath)                                            创建目录（支持多级目录）
    copy(srcPath, destPath, isOverride)                      复制文件或目录（可选覆
盖）
    move(srcFile, destDir, isOverride)                       移动文件或目录
*/

```

进程:(正在运行的程序)

程序：一份静态的代码（比如一个软件安装包）

进程：这段代码被运行起来之后的“活动状态”

进程 = 程序 + 运行环境 + 当前状态

进程的特性:

动态性： 进程是“正在运行”的

独立性： 每个进程有自己的内存空间

并发性： 多个进程可以同时运行（看起来同时）

生命周期： 创建 → 运行 → 结束

进程和线程的对比: 进程 = 一家公司,线程 = 公司里的员工

进程里面可以同时执行多个任务,每个任务就是线程

并行与并发的比较:

并发:单核运行，看起来"同时在做",但可能是**交替执行**.并发不一定同时，不需要多核

并行:多核运行,真正"同时在做",是**物理上的同时执行**.并行需要同时，需要多核

Java开启线程的方式:

- 继承**Thread**类(不推荐),扩展性不好，不能多继承
- 实现**Runnable**接口(推荐),扩展性好，它可以多继承，因为它是一个接口
- 实现**Callable**接口,如果线程任务需要有返回值，可以用Callable接口

线程相关方法:

```
/*
    线程设置名字和获取名字

    Thread类的方法：

    public String getName() : 获取线程名字
    public void setName() : 设置线程名字
    public static Thread currentThread() : 获取当前线程的对象

*/
```

```
/*
    休眠线程的方法

    public static void sleep(long time) : 让线程休眠指定的时间，单位为毫秒

*/
```

```
/*
    线程优先级的方法：

    public setPriority(int newPriority) : 设置线程优先级
    public final int getPriority() : 获取线程优先级

*/
```

```
/*
    public final void setDaemon(boolean on) : 设置为守护线程

*/
```

同步代码块 `synchronized`:

- 锁对象可以是任意对象,但是需要保证多条线程的锁对象,是同一把锁
- 同步可以解决多线程的数据安全问题,但是也会降低程序的运行效率

同步方法:

- 在方法的返回值类型前面加入 `synchronized` 关键字
- 方法分为静态和非静态
- 静态方法的锁对象是字节码对象,非静态方法的锁对象是this

lock锁:

```
Lock lock = new ReentrantLock();

lock.lock();    // 加锁
try {
    // 业务代码（临界区）
} finally {
    lock.unlock(); // 解锁（一定要写）
}
```

线程池:

作用:

1. 线程复用

不用每次都 new Thread

2. 控制线程数量

避免系统崩溃 (比如最多 10 个线程)

3. 任务排队

线程忙不过来时, 任务先排队

```
import java.util.concurrent.*;

public class TestPool {

    public static void main(String[] args) {

        ExecutorService pool = new ThreadPoolExecutor(
            2, // 核心线程数
            4, // 最大线程数
            60, TimeUnit.SECONDS, // 空闲存活时间
            new LinkedBlockingQueue<>(10) // 任务队列
        );

        pool.execute(() -> {
            System.out.println("任务1: " + Thread.currentThread().getName());
        });

        pool.shutdown();
    }
}
```

线程池的工作原理:

- 线程数 < corePoolSize → 直接创建线程
- 线程满了 → 放入队列
- 队列满了 → 创建新线程 (最多到 maxPoolSize)
- 再满 → 执行拒绝策略

自定义线程池:

```
import java.util.concurrent.*;

public class CustomThreadPool {

    public static void main(String[] args) {

        ThreadPoolExecutor pool = new ThreadPoolExecutor(
            2, // 核心线程数
```



```

        4, // 最大线程数
        60, // 空闲线程存活时间
        TimeUnit.SECONDS,
        new ArrayBlockingQueue<>(10), // 有界队列（重点）
        Executors.defaultThreadFactory(), // 线程工厂
        new ThreadPoolExecutor.AbortPolicy() // 拒绝策略
    );

    for (int i = 0; i < 5; i++) {
        int task = i;
        pool.execute(() -> {
            System.out.println(Thread.currentThread().getName()
                               + " 执行任务: " + task);
        });
    }

    pool.shutdown();
}
}

```

七大参数:

```

import java.util.concurrent.*;

/**
 * 自定义线程池示例
 */
public class CustomThreadPoolDemo {

    public static void main(String[] args) {

        /**
         * ThreadPoolExecutor 七大参数详解:
         */
        ThreadPoolExecutor executor = new ThreadPoolExecutor(

            /**
             * 1. corePoolSize（核心线程数）
             * -----
             * 👉 线程池中“长期存活”的线程数量
             * 👉 即使空闲也不会被销毁（默认情况）
             * 👉 这些线程优先处理任务
             *
             * 类比：公司正式员工数量
             */
            2,

            /**
             * 2. maximumPoolSize（最大线程数）
             * -----
             * 👉 线程池允许创建的最大线程数量
             * 👉 当核心线程 + 队列都满了，才会扩展线程
             *

```

```

    * 类比：正式员工 + 临时工总人数上限
    */
4,

/**
 * 3. keepAliveTime（空闲存活时间）
 * -----
 * 👉 非核心线程空闲多久会被回收
 * 👉 超过这个时间没有任务就销毁线程
 *
 * 类比：临时工多久没活就解雇
 */
60,

/**
 * 4. unit（时间单位）
 * -----
 * 👉 keepAliveTime 的时间单位
 */
TimeUnit.SECONDS,

/**
 * 5. workQueue（任务队列）★重点
 * -----
 * 👉 存放等待执行任务的队列
 * 👉 当核心线程满了，任务先进入这里排队
 *
 * ⚠ 推荐使用“有界队列”，防止内存爆炸
 *
 * 类比：任务排队大厅
 */
new ArrayBlockingQueue<>(3),

/**
 * 6. threadFactory（线程工厂）
 * -----
 * 👉 用来创建线程（可以自定义线程名）
 * 👉 方便排查日志和问题
 *
 * 类比：给员工起名字的HR
 */
new ThreadFactory() {
    private int count = 1;

    @Override
    public Thread newThread(Runnable r) {
        return new Thread(r, "my-thread-" + count++);
    }
},

/**
 * 7. handler（拒绝策略）★重点
 * -----

```

```

        * 👉 当线程池和队列都满了之后如何处理新任务
        *
        * 常见策略:
        * - AbortPolicy: 直接抛异常 (默认)
        * - CallerRunsPolicy: 让提交任务的线程自己执行
        * - DiscardPolicy: 直接丢弃任务
        * - DiscardOldestPolicy: 丢弃最早任务
        */
        new ThreadPoolExecutor.AbortPolicy()
    );

    /**
     * 提交任务测试
     */
    for (int i = 1; i <= 10; i++) {
        int taskId = i;

        executor.execute(() -> {
            System.out.println(Thread.currentThread().getName()
                + " 执行任务: " + taskId);

            try {
                Thread.sleep(1000); // 模拟任务执行
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        });
    }

    /**
     * 关闭线程池 (不是立刻停止, 会等任务执行完)
     */
    executor.shutdown();
}
}

```

2026年4月23日:

UDP:无连接协议

不管连没连接成功, 常用于在线视频, 传输快, 但是数据不安全易丢失, 1次64kb

TCP:连接协议

需要确保连接成功, 例如: 下载软件, 传输慢, 但是数据安全

三次握手, 四次挥手

三次握手: "1. 在吗? 2. 在的。 3. OK。"

1和3为客户端, 2为服务端

四次挥手: "1. 在吗? 2. 在的。 3. 断了。 4. OK。"

1和3为客户端, 2和4为服务端

TCP代码演示:

服务端:

```
import java.io.InputStream;
import java.net.ServerSocket;
import java.net.Socket;

/**
 * TCP服务端示例
 */
public class TCPServer {

    public static void main(String[] args) throws Exception {

        /**
         * 1. 创建 ServerSocket
         * -----
         * 👉 作用: 监听客户端连接
         * 👉 参数: 端口号 (必须和客户端一致)
         *
         * 类比: 开一个“服务窗口”等待客户来连接
         */
        ServerSocket serverSocket = new ServerSocket(8888);

        System.out.println("服务端启动, 等待连接...");

        /**
         * 2. accept() 阻塞等待客户端连接
         * -----
         * 👉 没有客户端连接时会一直阻塞
         * 👉 一旦有客户端连接, 就返回一个 Socket 对象
         *
         * 类比: 有人打电话进来了
         */
        Socket socket = serverSocket.accept();

        System.out.println("客户端已连接");

        /**
         * 3. 获取输入流 (读取客户端发来的数据)
         * -----
         * 👉 TCP 是基于流的通信
         */
        InputStream is = socket.getInputStream();

        byte[] buffer = new byte[1024];
        int len = is.read(buffer);

        System.out.println("收到消息: " + new String(buffer, 0, len));

        /**
         * 4. 关闭资源
         */
    }
}
```

```

        */
        socket.close();
        serverSocket.close();
    }
}

```

客户端:

```

import java.io.OutputStream;
import java.net.Socket;

/**
 * TCP客户端示例
 */
public class TCPClient {

    public static void main(String[] args) throws Exception {

        /**
         * 1. 创建 Socket
         * -----
         * 👉 需要指定服务器IP + 端口
         *
         * 类比: 拨打电话
         */
        Socket socket = new Socket("127.0.0.1", 8888);

        /**
         * 2. 获取输出流 (向服务端发送数据)
         */
        OutputStream os = socket.getOutputStream();

        os.write("你好, 我是客户端".getBytes());

        /**
         * 3. 关闭资源
         */
        socket.close();
    }
}

```

UDP的代码演示:

接收端:

```

import java.net.DatagramPacket;
import java.net.DatagramSocket;

/**
 * UDP接收端
 */
public class UDPReceiver {

```

```

public static void main(String[] args) throws Exception {

    /**
     * 1. 创建 DatagramSocket
     * -----
     * 🖱️ 绑定端口，监听数据
     *
     * 类比：开一个“收件箱”
     */
    DatagramSocket socket = new DatagramSocket(9999);

    /**
     * 2. 创建数据包用于接收数据
     */
    byte[] buffer = new byte[1024];

    DatagramPacket packet = new DatagramPacket(buffer, buffer.length);

    System.out.println("等待接收数据...");

    /**
     * 3. 接收数据（阻塞）
     */
    socket.receive(packet);

    /**
     * 4. 解析数据
     */
    String msg = new String(packet.getData(), 0, packet.getLength());

    System.out.println("收到消息: " + msg);

    socket.close();
}
}

```

发送端:

```

import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;

/**
 * UDP发送端
 */
public class UDPSender {

    public static void main(String[] args) throws Exception {

        /**
         * 1. 创建 DatagramSocket
         * -----

```

```

    * 👉 不需要指定端口（系统随机分配）
    *
    * 类比：准备寄信
    */
DatagramSocket socket = new DatagramSocket();

/**
 * 2. 准备数据
 */
byte[] data = "你好，我是UDP发送端".getBytes();

/**
 * 3. 创建数据包
 * -----
 * 👉 需要指定：
 * - 数据
 * - 接收方IP
 * - 接收方端口
 */
DatagramPacket packet = new DatagramPacket(
    data,
    data.length,
    InetAddress.getByName("127.0.0.1"),
    9999
);

/**
 * 4. 发送数据
 */
socket.send(packet);

socket.close();
}
}

```

枚举：一组固定、有限、不会改变的常量集合

- 一周只有：周一 ~ 周日
- 状态只有：成功 / 失败 / 处理中
- 性别只有：男 / 女

枚举基本定义：

```

enum Status {
    SUCCESS,
    FAIL,
    PROCESSING
}

```

枚举的使用方式：

```
public class TestEnum {
    public static void main(String[] args) {

        Status status = Status.SUCCESS;

        System.out.println(status); // 输出: SUCCESS
    }
}
```

带属性的枚举:

```
enum Status {

    SUCCESS(200, "成功"),
    FAIL(500, "失败");

    private int code;
    private String msg;

    // 构造器 (默认 private)
    Status(int code, String msg) {
        this.code = code;
        this.msg = msg;
    }

    public int getCode() {
        return code;
    }

    public String getMsg() {
        return msg;
    }
}
```

使用:

```
System.out.println(Status.SUCCESS.getCode()); // 200
```

常用方法:

```
Status.SUCCESS.name(); // "SUCCESS"
Status.SUCCESS.ordinal(); // 下标 (从0开始)
Status.valueOf("SUCCESS"); // 转换
```

反射:就是加载器, 在程序运行时, 动态获取类的信息, 并操作这个类

```
import java.lang.reflect.*;

class User {
    private String name;
```



```

    public void setName(String name) {
        this.name = name;
    }

    public String toString() {
        return "User{name='" + name + "'}";
    }
}

public class TestReflection {
    public static void main(String[] args) throws Exception {

        // 1. 获取 Class 对象
        Class<?> clazz = Class.forName("User");

        // 2. 创建对象
        Object obj = clazz.getDeclaredConstructor().newInstance();

        // 3. 获取方法
        Method method = clazz.getDeclaredMethod("setName", String.class);

        // 4. 调用方法
        method.invoke(obj, "张三");

        // 5. 输出
        System.out.println(obj);
    }
}

```

注解:给代码加标签, 让程序在运行或编译时做特殊处理

本质是接口.

```

import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME) // 运行时可用
@Target(ElementType.METHOD)       // 作用在方法上
public @interface MyAnnotation {
    String value();
}

```

使用注解:

```

public class Test {

    @MyAnnotation("测试方法")
    public void testMethod() {
        System.out.println("执行方法");
    }
}

```

用反射读取注解:

```
import java.lang.reflect.Method;

public class Main {
    public static void main(String[] args) throws Exception {

        Class<?> clazz = Test.class;

        Method method = clazz.getMethod("testMethod");

        if (method.isAnnotationPresent(MyAnnotation.class)) {
            MyAnnotation ann = method.getAnnotation(MyAnnotation.class);
            System.out.println(ann.value());
        }
    }
}
```

元注解:用来“修饰注解的注解”

给注解加注解

```
import java.lang.annotation.*;

/**
 * 自定义注解
 */
@Retention(RetentionPolicy.RUNTIME) // 运行时有效（可以反射读取）
@Target(ElementType.METHOD)       // 只能用在方法上
public @interface MyAnnotation {
    String value();
}
```

JS部分:

alert 弹窗:

```
alert("Hello JAVA")
```

let变量, const常量:

```
alert("Hello JAVA");
// let是变量
let age = 18;

// const是常量
const name = "Java";
```

数据类型

```
<!DOCTYPE html>
```

```

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>JS-数据类型</title>
</head>
<body>

  <script>
    //1. 数据类型
    // alert(typeof 10); //number
    // alert(typeof 1.5); //number

    // alert(typeof true); //boolean
    // alert(typeof false); //boolean

    // alert(typeof "Hello"); //string
    // alert(typeof 'JS'); //string
    // alert(typeof `JavaScript`); //string

    // alert(typeof null); //null ? -> object

    // let a ;
    // alert(typeof a); //undefined

    //2. 模板字符串 - 简化字符串拼接
    let name = 'Tom';
    let age = 18;

    console.log('我是'+name+', 我今年'+age+'岁');
    console.log(`我是${name}, 我今年${age}岁`);

  </script>
</body>
</html>

```

反引号的字符串拼接类似于Python经典的 `${变量名}`

foreach:

```

let arr = [1, 2, 3];

arr.forEach(item => {
  console.log(item);
});

```

函数:

```
//普通函数
function add(a,b){
    return a+b;
}
console.log(add(10,20));
//箭头函数
const add2 = (a,b) => a+b;
console.log(add2(30,40));
```

对象(Java中的方法)

```
//对象
let user = {
    name:"Tom",
    age:18,
    sayHello:function(){
        console.log(`Hello,我是${this.name},我今年${this.age}岁`);
    }
}
user.sayHello();
```

数组:

```
//数组
let arr = [1,2,3,4,5];
//取值
console.log(arr[0]); // 1
//遍历
for(let i=0;i<arr.length;i++){
    console.log(arr[i]);
}
//其他遍历方法,直接打印数组
console.log(arr)
//常用方法
arr.push(6); //添加元素
arr.pop(); //删除最后一个元素
```

Json:

```
// JSON
let jsonStr = '{"name":"Tom","age":18}';
let jsonObj = JSON.parse(jsonStr); //字符串转对象
console.log(jsonObj.name); // Tom
console.log(jsonObj.age); // 18
```

DOM(Document Object Model)文档对象模型

```
let h1 = document.querySelector('h1');
h1.innerHTML = '这是修改后的标题';
```

事件监听:让程序“等待某个行为发生，然后自动执行代码”

类似于一段待被激活的代码

语法:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>JS-引入方式</title>
  </head>
  <body>
    <input type="button" id="bt1" value="点击我" />
    <script>
      document.querySelector('#bt1').addEventListener('click',()=>{
        alert('按钮被点击了');
      })
    </script>
  </body>
</html>
```

常见事件:

- mouseenter(鼠标进入)
- mouseleave(鼠标离开)
- keydown (键盘按下)
- keyup(键盘抬起)
- focus (聚焦)
- blur (失焦)
- click (点击)
- input (输入)
- submit(表单提交时)
- change (变化)
- scroll (滚动)
- load (加载完成)

常见事件练习案例:

html页面代码:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>JS-引入方式</title>
</head>
```

```

<body>
  <form action="" style="text-align: center;">
    <input type="text" name="username" id="username">
    <input type="text" name="age" id="age">
    <input id="b1" type="submit" value="提交">
    <input id="b2" type="button" value="单击事件">
  </form>

  <br><br><br>

  <table width="800px" border="1" cellspacing="0" align="center">
    <tr>
      <th>学号</th>
      <th>姓名</th>
      <th>分数</th>
      <th>评语</th>
    </tr>
    <tr align="center">
      <td>001</td>
      <td>张三</td>
      <td>90</td>
      <td>很优秀</td>
    </tr>
    <tr align="center" id="last">
      <td>002</td>
      <td>李四</td>
      <td>92</td>
      <td>优秀</td>
    </tr>
  </table>
  <script type="module" src="JS/evento.js">
  </script>

</body>
</html>

```

引入的JS资源:

```

import { printLog } from './utils.js';

document.querySelector('#b2').addEventListener('click', () => {
  printLog("我被点击了...");
})

document.querySelector('#last').addEventListener('mouseenter', () => {
  printLog("鼠标移入了...");
})

document.querySelector('#last').addEventListener('mouseleave', () => {
  printLog("鼠标移出了...");
})

```

```

document.querySelector('#username').addEventListener('keydown', () => {
  printLog("键盘被按下了...");
})

document.querySelector('#username').addEventListener('keyup', () => {
  printLog("键盘被抬起了...");
}
)

document.querySelector('#age').addEventListener('blur', () => {
  printLog("失去焦点...");
})
document.querySelector('#age').addEventListener('focus', () => {

  printLog("获得焦点...");
})

document.querySelector('#age').addEventListener('input', () => {
  printLog("用户输入时触发...");
})
document.querySelector('form').addEventListener('submit', () => {
  alert("表单被提交了...");
}
)

```

JS资源所引入的工具类:

```

export function printLog(msg){
  console.log(msg);
}

```

Vue快速入门:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vue-快速入门</title>
</head>
<body>
  <div id="app">
    <h1>{{message}}</h1>
    <h1>{{count}}</h1>
  </div>
  <script type="module">
import { createApp } from 'https://unpkg.com/vue@3/dist/vue.esm-browser.js';
createApp({
  data(){
    return{
      message : 'Hello Vue',
      count : 100
    }
  }
})

```

```
}).mount('#app');  
</script>  
</body>  
</html>
```

Vue常用命令:

```
<!DOCTYPE html>  
<html>  
<head>  
  <meta charset="UTF-8">  
  <title>Vue常用指令示例</title>  
  <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>  
</head>  
  
<body>  
  <div id="app">  
  
    <!-- 1 v-model: 双向绑定 -->  
    <input v-model="username" placeholder="请输入用户名">  
    <p>用户名: {{ username }}</p>  
  
    <hr>  
  
    <!-- 2 v-bind (简写 : ): 绑定属性 -->  
      
    <p :title="tip">鼠标悬停看提示</p>  
  
    <hr>  
  
    <!-- 3 v-if / v-else: 条件渲染 -->  
    <button @click="show = !show">切换显示</button>  
  
    <p v-if="show">✓ 现在你能看到我 (v-if) </p>  
    <p v-else>✗ 我被隐藏了</p>  
  
    <hr>  
  
    <!-- 4 v-show: 控制显示 (只是 display:none) -->  
    <p v-show="show">我是 v-show 控制的</p>  
  
    <hr>  
  
    <!-- 5 v-for: 列表渲染 -->  
    <ul>  
      <li v-for="(item, index) in list" :key="index">  
        {{ index }} - {{ item }}  
      </li>  
    </ul>  
  
    <hr>  
  
    <!-- 6 v-on (@): 事件绑定 -->
```



```

<button @click="addItem">添加元素</button>

<hr>

<!-- 7 v-html: 解析HTML -->
<p v-html="htmlContent"></p>

<hr>

<!-- 8 v-text: 纯文本 -->
<p v-text="textContent"></p>

</div>

<script>
const { createApp } = Vue;

createApp({
  data() {
    return {
      // v-model 双向绑定
      username: "",

      // v-bind
      imgUrl: "https://vuejs.org/images/logo.png",
      tip: "这是 Vue Logo",

      // v-if / v-show
      show: true,

      // v-for
      list: ["Java", "Vue", "SpringBoot"],

      // v-html
      htmlContent: "<b style='color:red'>这是红色加粗HTML</b>",

      // v-text
      textContent: "这是纯文本内容"
    }
  },

  methods: {
    // v-on 事件方法
    addItem() {
      this.list.push("新技能 " + (this.list.length + 1));
      console.log("添加了一个元素");
    }
  }
}).mount("#app");
</script>

</body>
</html>

```

v-for:列表渲染

```
<tr v-for="(item,index) in items" :key="item.id">{{item}}</tr>
```

省略写法: `v-for="item in item"`

推荐在后面加 `key="item.id"`, 提升性能

v-bind和v-model:

- `v-bind`: 数据变页面变, 页面变数据不变
- `v-model`: 数据变页面变, 页面变数据变
- 插值表达式 `{{xx}}` 只能渲染数据, 不能在标签内部渲染属性, 渲染属性需要使用 `v-bind`

v-if:控制元素的显示与隐藏(一次性显示内容、登录状态、权限控制):

```
<td>
  <span v-if="e.job == 1">班主任</span>
  <span v-else-if="e.job == 2">讲师</span>
  <span v-else-if="e.job == 3">学工主管</span>
  <span v-else-if="e.job == 4">教研主管</span>
  <span v-else-if="e.job == 5">咨询师</span>
  <span v-else>其他</span>
</td>
```

- 如果页面不需要频繁的去渲染, 那就v-if

v-show:控制元素的显示与隐藏(菜单显示隐藏、Tab切换):

```
<!-- v-show: 控制元素的显示与隐藏 -->
<td>
  <span v-show="e.job == 1">班主任</span>
  <span v-show="e.job == 2">讲师</span>
  <span v-show="e.job == 3">学工主管</span>
  <span v-show="e.job == 4">教研主管</span>
  <span v-show="e.job == 5">咨询师</span>
</td>
```

`v-if` = 控制“是否存在”(销毁/创建)

`v-show` = 控制“是否显示”(隐藏/显示)

少切换用 `v-if`

多切换用 `v-show`

v-model:双向绑定

v-on:事件绑定

Ajax(Asynchronous JavaScript And XML:异步的JavaScript和XML):是一种“在不刷新页面的情况下, 向服务器请求数据”的技术。

- 是一种“在不刷新页面的情况下, 向服务器请求数据”的技术。

```
axios.get("/api/user")
  .then(res => {
    console.log(res.data);
  });
```

Ajax = 不刷新页面 + 异步请求数据 + 局部更新页面

Axios:对原生的Ajax进行了封装，简化书写，快速开发

GET请求:

```
axios.get('/api/user')
  .then(response => {
    console.log(response.data);
  })
  .catch(error => {
    console.log(error);
  });
```

POST请求:

```
axios.post('/api/login', {
  username: 'admin',
  password: '123456'
})
  .then(res => {
    console.log(res.data);
  });
```

完整示例:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Axios 示例</title>
  <!-- Vue -->
  <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>

  <!-- Axios -->
  <script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
</head>

<body>

<div id="app">

  <h2>Axios 获取用户数据</h2>

  <button @click="getUser">获取用户</button>

  <hr>
```

```

    <!-- 显示数据 -->
    <p>用户名: {{ user.name }}</p>
    <p>邮箱: {{ user.email }}</p>

</div>

<script>
const { createApp } = Vue;

createApp({
  data() {
    return {
      user: {}
    }
  },

  methods: {

    // 🔥 Axios GET 请求
    getUser() {

      axios.get("https://jsonplaceholder.typicode.com/users/1")
        .then(res => {

          // 把返回数据存入 vue
          this.user = res.data;

          console.log("请求成功: ", res.data);
        })
        .catch(err => {
          console.log("请求失败: ", err);
        });

    }
  }
}).mount("#app");
</script>

</body>
</html>

```

Axios-请求方式别名:

```

axios.[请求方式](url [,data [, config]])

```

```

    axios.get("https://jsonplaceholder.typicode.com/users/1")
      .then(res => {

        // 把返回数据存入 vue
        this.user = res.data;

        console.log("请求成功: ", res.data);
      })
      .catch(err => {
        console.log("请求失败: ", err);
      });

```

Axios综合案例:

```

<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <title>Tlias智能学习辅助系统</title>

  <style>
    /* ===== 导航栏样式 ===== */
    .navbar {
      background-color: #b5b3b3; /* 灰色背景 */
      display: flex; /* 使用flex布局 */
      justify-content: space-between; /* 左右分布 */
      padding: 10px; /* 内边距 */
      align-items: center; /* 垂直居中 */
    }

    .navbar h1 {
      margin: 0; /* 去掉默认外边距 */
      font-weight: bold;
      color: white;
      font-family: "楷体"; /* 设置字体 */
    }

    .navbar a {
      color: white;
      text-decoration: none; /* 去掉下划线 */
    }

    /* ===== 搜索表单 ===== */
    .search-form {
      display: flex;
      align-items: center;
      gap: 10px; /* 控件之间间距 */
      margin: 20px 0;
    }

    .search-form input,
    .search-form select {

```



```

</div>

<!-- 🔥 查询表单 -->
<form class="search-form">

  <!-- v-model: 双向绑定（输入框 → vue数据） -->
  <label>姓名: </label>
  <input type="text" v-model="searchForm.name" placeholder="请输入姓名">

  <label>性别: </label>
  <select v-model="searchForm.gender">
    <option value=""></option>
    <option value="1">男</option>
    <option value="2">女</option>
  </select>

  <label>职位: </label>
  <select v-model="searchForm.job">
    <option value=""></option>
    <option value="1">班主任</option>
    <option value="2">讲师</option>
    <option value="3">学工主管</option>
    <option value="4">教研主管</option>
    <option value="5">咨询师</option>
  </select>

  <!-- v-on / @: 事件绑定 -->
  <button type="button" @click="search">查询</button>
  <button type="button" @click="clear">清空</button>
</form>

<!-- 🔥 员工表格 -->
<table>

  <!-- 表头 -->
  <thead>
    <tr>
      <th>序号</th>
      <th>姓名</th>
      <th>性别</th>
      <th>头像</th>
      <th>职位</th>
      <th>入职日期</th>
      <th>更新时间</th>
      <th>操作</th>
    </tr>
  </thead>

  <!-- 表体 -->
  <tbody>

    <!-- v-for: 循环渲染员工列表 -->
    <tr v-for="(e, index) in empList" :key="e.id">

```

```

    <!-- index + 1: 序号 -->
    <td>{{ index + 1 }}</td>

    <!-- 插值表达式: 显示数据 -->
    <td>{{ e.name }}</td>

    <!-- 三元表达式 -->
    <td>{{ e.gender == 1 ? '男' : '女' }}</td>

    <!-- v-bind: 绑定图片地址 -->
    <td>
        
    </td>

    <!-- v-if: 条件渲染 -->
    <td>
        <span v-if="e.job == 1">班主任</span>
        <span v-else-if="e.job == 2">讲师</span>
        <span v-else-if="e.job == 3">学工主管</span>
        <span v-else-if="e.job == 4">教研主管</span>
        <span v-else-if="e.job == 5">咨询师</span>
        <span v-else>其他</span>
    </td>

    <td>{{ e.entrydate }}</td>
    <td>{{ e.updatetime }}</td>

    <!-- 操作按钮 -->
    <td>
        <button type="button">编辑</button>
        <button type="button">删除</button>
    </td>

</tr>
</tbody>
</table>

<!-- 页脚 -->
<footer class="footer">
    <p>江苏传智播客教育科技股份有限公司</p>
</footer>

</div>

<!-- 引入 axios -->
<script src="JS/axios.js"></script>

<!-- vue3 模块化写法 -->
<script type="module">
    import { createApp } from 'https://unpkg.com/vue@3/dist/vue.esm-browser.js'

    createApp({

```



```

/* ===== 数据 ===== */
data() {
  return {
    // 查询条件（绑定表单）
    searchForm: {
      name: '',
      gender: '',
      job: ''
    },

    // 员工列表数据（接口返回）
    empList: []
  }
},

/* ===== 方法 ===== */
methods: {

  // 🔥 查询员工（核心）
  async search() {

    // axios 发送 GET 请求（带参数）
    let result = await axios.get(
      `https://web-server.itheima.net/emps/list
      ?name=${this.searchForm.name}
      &gender=${this.searchForm.gender}
      &job=${this.searchForm.job}`
    );

    // 把返回数据赋值给 empList（触发页面更新）
    this.empList = result.data.data;
  },

  // 🔥 清空条件
  clear() {
    this.searchForm.name = '';
    this.searchForm.gender = '';
    this.searchForm.job = '';

    // 重新查询
    this.search();
  }
},

/* ===== 生命周期钩子 ===== */
mounted() {
  // 页面加载完成自动查询一次
  this.search();
}

}).mount('#container')
</script>

```

```
</body>
</html>
```

钩子函数:在特定生命周期自动执行的函数

```
mounted(){
  this.search();
}
```

mounted = 页面已经渲染完成 → 可以安全请求数据并更新页面

钩子方法(函数)案例:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Vue生命周期钩子演示</title>
  <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
</head>

<body>

<div id="app">
  <h2>{{ message }}</h2>

  <button @click="change">修改数据</button>
  <button @click="destroy">销毁组件</button>
</div>

<script>
const { createApp } = Vue;

const app = createApp({

  data() {
    return {
      message: "Hello Vue 生命周期"
    }
  },

  // ===== 创建阶段 =====

  beforeCreate() {
    console.log("1 beforeCreate: 数据还没初始化");
  },

  created() {
    console.log("2 created: 数据已初始化");
  },
```

```

// ===== 挂载阶段 =====

beforeMount() {
  console.log("3 beforeMount: 即将渲染DOM");
},

mounted() {
  console.log("4 mounted: 页面已经渲染完成");

  // 👉 最常用: 发送请求
  console.log("这里最适合发请求");
},

// ===== 更新阶段 =====

beforeUpdate() {
  console.log("5 beforeUpdate: 数据变了, DOM还没更新");
},

updated() {
  console.log("6 updated: DOM已经更新");
},

// ===== 卸载阶段 =====

beforeUnmount() {
  console.log("7 beforeUnmount: 组件即将销毁");
},

unmounted() {
  console.log("8 unmounted: 组件已销毁");
},

methods: {
  change() {
    this.message = "数据被修改了";
  },

  destroy() {
    // 手动卸载Vue应用
    app.unmount();
  }
}
});

app.mount("#app");
</script>

</body>
</html>

```

Maven:是一个 Java 项目管理和构建工具，用来管理依赖、编译、打包项目。

- 传统开发很痛苦:手动下载 jar 包,手动放到项目里,版本冲突难处理,编译、打包要自己写脚本
- 有了Maven:自动下载依赖,自动管理版本,一键编译、打包、统一项目结构

Maven功能:

1.依赖管理:只需要写配置,Maven自动帮你下载jar包

2.项目构建:一条命令搞定,自动编译代码、运行测试、打成jar/war

```
mvn package
```

Maven常用命令:

```
mvn clean      # 清理
mvn compile    # 编译
mvn test       # 测试
mvn package    # 打包
mvn install    # 安装到本地仓库
```

Maven = 依赖管理 + 项目构建 + 标准化开发

IDEA里Maven的操作:

- 点开IDEA里的Maven,在Lifecycle里有编译 `compile`, 打包 `package` 等操作
- 在Dependencies里是通过在pom.xml文件里写的所下载的jar包

Maven中pom.xml文件整体的结构:

```
<?xml version="1.0" encoding="UTF-8"?>

<!-- Maven 项目根标签 -->
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

  <!-- XML 约束文件（校验 pom 结构） -->
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <!-- 模型版本（固定写 4.0.0） -->
  <modelVersion>4.0.0</modelVersion>

  <!-- ===== 项目基本信息 ===== -->

  <!-- 组织/公司标识（一般用包名反写） -->
  <groupId>com.example</groupId>

  <!-- 项目名称（唯一标识） -->
  <artifactId>demo-project</artifactId>

  <!-- 项目版本 -->
  <version>1.0.0</version>
```

```
<!-- 打包方式: jar / war -->
<packaging>jar</packaging>

<!-- ===== 继承 SpringBoot 父工程 ===== -->
<!-- 可以帮你管理依赖版本（非常重要） -->
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.7.5</version>
</parent>

<!-- ===== 属性配置 ===== -->
<properties>
    <!-- Java版本 -->
    <java.version>1.8</java.version>

    <!-- 编码格式 -->
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<!-- ===== 依赖管理 ===== -->
<dependencies>

    <!-- 🔥 web开发依赖（最核心） -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- 🔥 测试依赖 -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>

        <!-- test 表示只在测试阶段使用 -->
        <scope>test</scope>
    </dependency>

    <!-- 🔥 MySQL驱动 -->
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <version>8.0.33</version>
    </dependency>

    <!-- 🔥 Lombok（简化代码） -->
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
```

```

        <!-- provided: 编译时需要, 运行时不需要 -->
        <scope>provided</scope>
    </dependency>

</dependencies>

<!-- ===== 构建配置 ===== -->
<build>

    <!-- 插件 -->
    <plugins>

        <!-- 🔥 SpringBoot 打包插件 -->
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>

            <!-- 可以打成可运行jar -->
        </plugin>

    </plugins>

</build>

</project>

```

Maven的三类仓库:本地仓库、中央仓库(Maven团队)、远程仓库

Maven的安装太繁琐了就不写了,别忘了还有IDEA的配置

Maven的坐标(写法类似于html的标签):用来唯一定位一个 Java 项目/依赖的“地址”

现实世界: 省 + 市 + 门牌号

Maven世界: groupId + artifactId + version

- groupId:组织或公司
- artifactId:项目名(模块名)
- version:版本号

案例(SpringBoot):

```

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <version>3.2.0</version>
</dependency>

```

在IDEA中导入Maven项目:

- 第一种方法:项目结构-->modules-->加号-->导入pom文件
- 第二种方法:点击IDEA侧边栏的M的Maven图标, 点击加号导入pom文件

Maven的依赖结构:

```
<dependency>
  <groupId>依赖所属组织</groupId>
  <artifactId>依赖名称</artifactId>
  <version>版本号</version>
</dependency>
```

真实的例子:

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.0.33</version>
</dependency>
```

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

测试依赖(scope):

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <scope>provided</scope>
</dependency>
```

dependency的scope使用:

- **compile(默认):**全部阶段可用
- **test:**只在测试用
- **provided:**编译用,运行不用
- **runtime:**运行时用

排除依赖:exclusions(详细使用在网络上查找)

Maven的生命周期(Maven 帮你规定好的一套“从代码到运行/发布”的标准流程):

1.Clean(清理)生命周期阶段:

- **pre-clean:**清理前准备
- **clean:**删除target

- post-clean:清理后处理

2.Default(构建项目)生命周期阶段:

- validate:校验项目结构
- compile:编译代码
- test:执行测试
- package:打包(jar/war)
- install:安装到本地仓库
- deploy:发布到远程仓库

3.Site(生产项目文档，了解即可)生命周期阶段:

- site:生成站点文档
- deploy-site:发布文档

真实开发流程:

```
写代码
↓
mvn clean
↓
mvn compile
↓
mvn test
↓
mvn package
↓
mvn install
```

形象比喻:

```
validate → 检查原料
compile → 加工零件
test → 质量检测
package → 装箱
install → 入库
deploy → 发货
```

Maven 有三套生命周期：Clean（清理）、Default（构建）、Site（文档），其中 Default 生命周期最重要，包含编译、测试、打包、安装等核心流程。

后面的阶段都是依据于前面的阶段的

测试的分类:

- 单元测试(白盒测试):对**最小功能单元（方法/函数**进行测试
- 集成测试(灰盒测试):测试多个模块之间是否能正常协作
- 系统测试(黑盒测试):整个系统是否按需求正常运行
- 验收测试(黑盒测试):由用户或客户验证系统是否满足需求

白盒测试(内部):测试逻辑,黑盒测试(外部):测试功能,

灰盒测试(内部与外部):既测试逻辑,又测试功能

单元测试:对程序中“最小功能单元”(通常是方法)进行正确性验证的测试方式

JUnit:最流行的Java测试框架之一,方便程序进行单元测试

- 测试代码与源代码分开,便于维护
- 可以根据需要进行自动化测试
- 可自动分析测试结果,产出测试报告

1.首先在pom.xml文件中引入JUnit:

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.9.1</version>
  </dependency>
</dependencies>
```

2.test/java目录下,创建测试类,并编写对应的测试方法,并在方法上声明@Test注解

```
package org.example;

import org.junit.jupiter.api.Test;

public class UserServiceTest {

    @Test
    public void test(){
        UserService userService = new UserService();
        Integer age = userService.getAge("102302233202021042");
        System.out.println(age);
    }

    @Test
    public void testGetGender(){
        UserService userService = new UserService();
        String gender = userService.getGender("401203330020201342");
        System.out.println(gender);
    }

}
```

Java中能运行,不能代表代码没有问题,比如值是否跟我们预期想要的是一样的

断言:JUnit提供了一些辅助方法,用来帮助我们确定被测试的方法是否按照预期的效果正常工作,这种方式被称为断言

```
// CalculatorTest.java
import org.junit.jupiter.api.Test;
```

```
import static org.junit.jupiter.api.Assertions.*;

public class CalculatorTest {

    Calculator c = new Calculator();

    // ===== 基本断言 =====

    @Test
    public void testAdd() {
        // 断言: 期望值 = 5, 实际值 = 方法结果
        assertEquals(5, c.add(2, 3), "加法计算错误");
    }

    @Test
    public void testIsEven() {
        // 断言 true
        assertTrue(c.isEven(4), "4 应该是偶数");

        // 断言 false
        assertFalse(c.isEven(5), "5 不应该是偶数");
    }

    // ===== 空值断言 =====

    @Test
    public void testNull() {
        String str = null;

        // 判断是否为 null
        assertNull(str, "应该为 null");

        // 判断不为 null
        assertNotNull(c.getName(), "返回值不应该为 null");
    }

    // ===== 字符串断言 =====

    @Test
    public void testString() {
        // 判断字符串是否相等
        assertEquals("Tom", c.getName(), "名字不匹配");
    }

    // ===== 数组断言 =====

    @Test
    public void testArray() {
        int[] expected = {1, 2, 3};
        int[] actual = {1, 2, 3};

        // 判断数组是否相同
        assertEquals(expected, actual, "数组不一致");
    }
}
```

```

}

// ===== 异常断言（重点🔥） =====

@Test
public void testException() {
    // 断言会抛出 ArithmeticException（除0异常）
    assertThrows(ArithmeticException.class, () -> {
        c.divide(10, 0);
    }, "没有抛出除零异常");
}

// ===== 组合断言 =====

@Test
public void testAll() {
    assertAll("组合测试",
        () -> assertEquals(5, c.add(2, 3)),
        () -> assertTrue(c.isEven(6)),
        () -> assertEquals("Tom", c.getName())
    );
}
}

```

JUnit常见注解:

```

import org.junit.jupiter.api.*;

public class UserServiceTest {

    // ===== 生命周期注解 =====

    @BeforeAll
    static void beforeAll() {
        // 在所有测试方法执行之前，只执行一次（必须是 static）
        System.out.println("🔥 所有测试开始之前执行（初始化环境）");
    }

    @AfterAll
    static void afterAll() {
        // 在所有测试方法执行之后，只执行一次
        System.out.println("🔥 所有测试结束之后执行（释放资源）");
    }

    @BeforeEach
    void beforeEach() {
        // 每个测试方法执行之前都会执行
        System.out.println("👉 每个测试前执行");
    }

    @AfterEach
    void afterEach() {
        // 每个测试方法执行之后都会执行
    }
}

```

```

        System.out.println("👉 每个测试后执行");
    }

    // ===== 测试方法 =====

    @Test
    void testAdd() {
        System.out.println("测试加法逻辑");
    }

    @Test
    void testDelete() {
        System.out.println("测试删除逻辑");
    }

    // ===== 禁用测试 =====

    @Disabled("暂时不测试这个方法")
    @Test
    void testUpdate() {
        System.out.println("这个不会执行");
    }

    // ===== 重复测试 =====

    @RepeatedTest(3)
    void testRepeat() {
        System.out.println("重复执行测试");
    }

    // ===== 参数化测试 =====

    @ParameterizedTest
    @ValueSource(ints = {1, 2, 3, 4})
    void testParam(int num) {
        System.out.println("参数是: " + num);
    }

    // ===== 显示名称 =====

    @DisplayName("自定义测试名称")
    @Test
    void testName() {
        System.out.println("测试名称展示");
    }

    // ===== 超时测试 =====

    @Test
    @Timeout(2) // 超过2秒就失败
    void testTimeout() throws InterruptedException {
        Thread.sleep(1000); // 模拟耗时操作
        System.out.println("未超时");
    }

```

```
}  
}
```

两个Before注解主要用来做环境的准备工作

两个After注解主要用来做资源的释放工作

JUnit的参数化测试:

```
import org.junit.jupiter.params.ParameterizedTest;  
import org.junit.jupiter.params.provider.ValueSource;  
import org.junit.jupiter.params.provider.CsvSource;  
  
import static org.junit.jupiter.api.Assertions.*;  
  
public class CalculatorParamTest {  
  
    calculator calculator = new Calculator();  
  
    // ===== 1. 单参数测试 =====  
    @DisplayName("这是单元测试修改后的名字")  
    @ParameterizedTest  
    @ValueSource(ints = {2, 4, 6, 8, 10})  
    void testIsEven(int num) {  
        // 每次传入一个参数  
        assertTrue(calculator.isEven(num));  
    }  
  
    // ===== 2. 单参数（反例） =====  
    @DisplayName("这是单元测试修改后的名字2")  
    @ParameterizedTest  
    @ValueSource(ints = {1, 3, 5, 7})  
    void testIsNotEven(int num) {  
        assertFalse(calculator.isEven(num));  
    }  
  
    // ===== 3. 参数 + 期望值 =====  
    @DisplayName("这是单元测试修改后的名字3")  
    @ParameterizedTest  
    @CsvSource({  
        "2, 4",  
        "3, 9",  
        "5, 25",  
        "10, 100"  
    })  
    void testSquare(int input, int expected) {  
        // input → 输入值  
        // expected → 期望结果  
        assertEquals(expected, calculator.square(input));  
    }  
}
```

其他案例:

```

package org.example;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;

public class UserServiceTest {
    @DisplayName("测试性别A1")
    @Test
    public void test(){
        UserService userService = new UserService();
        Integer age = userService.getAge("102302233202021042");
        System.out.println(age);
    }

    @DisplayName("测试性别A2")
    @Test
    public void testGetGender(){
        UserService userService = new UserService();
        String gender = userService.getGender("401203330020201342");
        System.out.println(gender);
    }

    /**
     * 参数化测试
     */
    @DisplayName("参数化测试")
    @ParameterizedTest
    @ValueSource(strings = {"401203330020201342", "401203330020201322", "211203330020201342"})
    public void testGetGender2(){
        UserService userService = new UserService();
        Integer age = userService.getAge("102302233202021042");
        System.out.println(age);
    }
}

```

Run xxx with Coverage :运行并查看覆盖率

覆盖率可以查看类的测试、方法的测试、代码的测试、分支的测试覆盖率

Maven的依赖范围

1. compile: 编译时依赖，默认范围。在编译、测试和运行时都有效，会打包到最终的构件中。
2. provided: 已提供范围。在编译和测试时有效，但在运行时由容器或JDK提供，不会打包到最终构件中。例如：servlet-api。
3. runtime: 运行时范围。在测试和运行时有效，但编译时不需要。例如：JDBC驱动实现。
4. test: 测试范围。仅在测试编译和执行时有效，不会打包到最终构件中。例如：JUnit。
5. system: 系统范围。与provided类似，但需要显式提供本地文件系统中的jar包路径，不推荐使用。
6. import: 导入范围。仅用于<dependencyManagement>部分，用于导入其他POM文件的依赖管理配置。

Maven常见问题:因为网络问题没有下载成功, Maven会留下残留

1. 删除Maven的缓存文件:

- 找到本地仓库目录 (通常在 `~/.m2/repository`)
- 删除包含 `.lastUpdated` 文件的目录
- 或者使用命令清理: `mvn dependency:purge-local-repository`

2. 重新下载依赖:

- 在 IDEA 中右键项目 → Maven → Reload Project
- 或使用命令行: `mvn clean install -U` (-U 表示强制更新快照和发布版本)

3. 配置镜像源加速下载:

- 在 `settings.xml` 中配置阿里云镜像:

```
<mirrors>
  <mirror>
    <id>aliyunmaven</id>
    <mirrorOf>*</mirrorOf>
    <name>阿里云公共仓库</name>
    <url>https://maven.aliyun.com/repository/public</url>
  </mirror>
</mirrors>
```

4. 检查网络连接和代理设置

5. 手动删除有问题的依赖目录后重新构建项目

SpringBoot:是一个用来“快速开发 Java 后端项目”的框架, 让你不用写大量配置就能直接运行 Spring 应用

之前写项目需要配置许多(XML、Tomcat、依赖一堆)

SpringBoot可以自动配置, 不需要我们配置, 只需要写代码, SpringBoot会自动完成很多配置

SpringBoot入门程序:

- 1.创建SpringBoot项目
- 2.编写Controller类
- 3.编写启动类(或者用已有的启动类)
- 4.运行项目

HelloController.java(Controller类)

```

package org.example.springbootdemo;

import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController // 表示当前是一个请求处理类
public class HelloController {
    @RequestMapping("/hello")
    public String hello(String name){
        System.out.println("name:"+name);
        return "Hello " + name + "~";
    }
}

```

SpringBootDemoApplication.java(启动类)

```

package org.example.springbootdemo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringBootDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringBootDemoApplication.class, args);
    }

}

```

@RestController: 表示当前是一个请求处理类，处理所有的请求，返回的数据都是json格式

@RequestMapping: 表示当前类处理请求的路径，可以指定多个路径

@SpringBootApplication: 启动类注解，表示当前是一个SpringBoot项目

如果创建Module时,Server URL(骨架)连不上，就用国内阿里云的镜像去连

start.aliyun.com

HTTP协议:是浏览器和服务端之间“传递数据的规则”（通信协议）

HTTP请求协议的数据格式

1. 请求行: 请求数据的第一行(请求方式 请求路径 HTTP/1.1)
2. 请求头:第二行开始,格式key:value
3. 请求体:如果是POST请求,则存放请求参数

SpringBoot请求协议的数据获取:

```

package org.example.springbootdemo;

import jakarta.servlet.http.HttpServletRequest;
import org.springframework.web.bind.annotation.RequestMapping;

```



```

import org.springframework.web.bind.annotation.RestController;

@RestController
public class RequestController {

    @RequestMapping("/request")
    public String request(HttpServletRequest request){
        // 1. 获取请求方式
        String method = request.getMethod(); //GET
        System.out.println("请求方式:"+ method);

        // 2. 获取请求url地址
        String url = request.getRequestURI().toString(); // localhost:8080/requests
        System.out.println("请求url地址:"+ url);
        String uri = request.getRequestURI(); // /request
        System.out.println("请求uri地址:"+ uri);

        // 3. 获取请求协议
        String protocol = request.getProtocol(); // HTTP/1.1
        System.out.println("请求协议:"+protocol);

        // 4. 获取请求参数 - name,age
        String name = request.getParameter("name");
        String age = request.getParameter("age");
        System.out.println("name:"+name+", age:"+age);

        // 5. 获取请求头 - Accept
        String accept = request.getHeader("Accept");
        // 如果需要获取Cookie, 则把Accept改一下就行

        return "OK";
    }
}

```

浏览器访问 `http://localhost:8080/request?name=shadown7z&age=18`

可以从代码看出 `@RequestMapping("/request")` 是请求路径的设置

而 `HttpServletRequest` 对象里面封装了所有的请求信息, 直接调用 `request.xx` 就能获取请求数据的内容

再一个就是 `?` 后面是请求附带的内容数据, `&` 表连接, 连接前后数据

HTTP响应协议的数据格式:

1. 响应行: 响应数据第一行(协议、状态码、描述)
2. 第二行开始, 格式key:value
3. 响应体: 最后一部分, 存放响应数据

HTTP响应的状态码:

1. 1xx 信息提示
2. 2xx 请求成功
3. 3xx 重定向
4. 4xx 客户端错误(客户端的问题)

5. 5xx 服务器错误

SpringBoot响应协议的数据获取:

使用封装的 `HttpServletResponse` 对象

SpringBoot的响应协议数据代码:

```
package org.example.springbootdemo;

import jakarta.servlet.http.HttpServletResponse;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.io.IOException;

@RestController
public class ResponseController {

    // 方式一：基于原始的HttpServletResponse封装
    @RequestMapping("/response")
    public void response(HttpServletResponse response) throws IOException {
        // 1. 响应状态码
        response.setStatus(401);

        // 2. 设置响应头
        response.setHeader("name", "shadown7z");

        // 3. 设置响应体
        response.getWriter().write("<h1>Hello world</h1>");
    }

    /**
     * 方式二：使用ResponseEntity - Spring中封装的响应对象
     */
    @RequestMapping("/response2")
    public ResponseEntity<String> response2(){
        return ResponseEntity.status(401) // 1. 设置响应状态码
            .header("name", "shadown7z") // 2. 添加响应头
            .body("<h1>Hello Shadown7z</h1>"); // 3. 添加响应体
    }
}
```

通常情况下响应状态码不需要我们自己设置，SpringMVC会自动设置响应状态码，比如返回200，400，500等。

SpringBootWeb案例:(多看阶段3第四章的SpringBootWeb案例)

写封装类 User.java

```
package com.shadow.spirngbotweb01.pojo;

import lombok.AllArgsConstructor;
```

```

import Lombok.Data;
import Lombok.NoArgsConstructor;

import java.time.LocalDateTime;

/**
 * 用户信息
 */
@Data
@NoArgsConstructor
@AllArgsConstructor
public class User {
    private Integer id;
    private String username;
    private String password;
    private String name;
    private Integer age;
    private LocalDateTime updateTime;
}

```

写UserController.java,Controller类:

```

package com.shadow.spirngbotweb01.controller;

import cn.hutool.core.io.IoUtil;
import com.shadow.spirngbotweb01.pojo.User;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.InputStream;
import java.nio.charset.StandardCharsets;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

/**
 * 用户信息的Controller
 */
@RestController
public class UserController {
    @RequestMapping("/list")
    public List<User> list() throws Exception {
        // 1. 加载并读取user.txt文件, 获取用户数据
        //InputStream in = new
        FileInputStream("D:\\Java\\Codewordspace\\untitled\\untitled\\spirngbot-web-
01\\src\\main\\resources\\user.txt");
        InputStream in = this.getClass().getClassLoader().getResourceAsStream("user.txt");
    }
}

```

```

        ArrayList<String> lines = IoUtil.readLines(in, StandardCharsets.UTF_8, new
ArrayList<>());

        // 2. 解析用户信息, 封装成User对象 -> list集合
        List<User> userList = lines.stream().map(line -> {
            String[] parts = line.split(",");
            Integer id = Integer.parseInt(parts[0]);
            String username = parts[1];
            String password = parts[2];
            String name = parts[3];
            Integer age = Integer.parseInt(parts[4]);
            LocalDateTime updateTime = LocalDateTime.parse(parts[5],
DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss"));
            return new User(id, username, password, name, age, updateTime);
        }).toList();

        // 3. 返回数据(json)
        return userList;
    }
}

```

1. 获取数据,
2. 封装数据,
3. 返回数据(json)

@ResponseBody 注解的作用:

- 将controller方法的返回返回值转为json格式的数据
- 如果是对象或集合, 会先转为json, 再响应
- @RestController = @Controller + @ResponseBody

以上的代码要反复敲好几遍, 静态资源在配套资料里

分层解耦-三层架构:

- controller: 接收和响应数据
- service: 逻辑处理
- dao: 数据访问(Data Access Object), 负责数据访问操作

为什么要分成解耦? 因为这样复用性高、易维护

controller调用service, service调用dao

controller --> service ---> dao

UserController.java

```

package com.shadow.spirngbotweb01.controller;

import cn.hutool.core.io.IoUtil;
import com.shadow.spirngbotweb01.pojo.User;
import com.shadow.spirngbotweb01.service.UserService;

```

```

import com.shadow.spirngbotweb01.service.impl.UserServiceImpl;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.InputStream;
import java.nio.charset.StandardCharsets;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

/**
 * 用户信息的Controller
 */
@RestController
public class UserController {

    private UserService userService = new UserServiceImpl();

    @RequestMapping("/list")
    public List<User> list() throws Exception {
        // 1. 调用service获取数据
        List<User> userList = userService.findAll();

        // 2. 返回数据(json)
        return userList;
    }
}

```

UserServiceImpl.java

```

package com.shadow.spirngbotweb01.pojo;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

/**
 * 用户信息
 */
@Data
@NoArgsConstructor
@AllArgsConstructor
public class User {
    private Integer id;
}

```

```

private String username;
private String password;
private String name;
private Integer age;
private LocalDateTime updateTime;
}

```

UserDaoImpl.java

```

package com.shadow.spirngbotweb01.dao.impl;

import cn.hutool.core.io.IoUtil;
import com.shadow.spirngbotweb01.dao.UserDao;

import java.io.InputStream;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.List;

public class UserDaoImpl implements UserDao {
    @Override
    public List<String> findAll(){
        // 1. 加载并读取user.txt文件，获取用户数据
        InputStream in = this.getClass().getClassLoader().getResourceAsStream("user.txt");
        ArrayList<String> lines = IoUtil.readLines(in, StandardCharsets.UTF_8, new
ArrayList<>());
        return lines;
    }
}

```

接口代码没有写，因为没有什么代码，只是定义了一个规范

分三层架构，就是 controller --> service --> dao ,这样子调用的

分层之后的解耦操作:

- 耦合:衡量软件中各个层/各个模块的依赖关联程度
- 内聚:软件中各个功能模块内部的功能联系

内聚: 一个部门是否只做一件事

耦合: 部门之间是否互相依赖过多

软件设计的黄金原则:低耦合+高内聚

耦合看"模块之间"

内聚看"模块内部"

IOC(Inversion Of Control)控制反转和DI(Dependency Injection)依赖注入:

- **IOC:**(对象操作控制权交给容器)控制反转，反转控制，反转对象，反转对象实例，反转对象创建过程
- **DI:**(注入所需的依赖对象)依赖注入，注入对象，注入对象实例，注入对象创建过程
- **Bean对象:** IOC容器中创建、管理的对象，被称之为**Bean**

Repository 层 (数据层) :

```
package com.example.demo.repository;

import org.springframework.stereotype.Repository;

@Repository // 交给IOC容器管理
public class UserRepository {

    public String getUserById(Long id) {
        return "User{id=" + id + ", name=Tom}";
    }
}
```

Service 层 (业务层) :

```
package com.example.demo.service;

import com.example.demo.repository.UserRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service // 交给IOC容器管理
public class UserService {

    // DI: 依赖注入
    @Autowired
    private UserRepository userRepository;

    public String getUser(Long id) {
        return userRepository.getUserById(id);
    }
}
```

Controller 层 (接口层) :

```
package com.example.demo.controller;

import com.example.demo.service.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/user")
public class UserController {

    @Autowired
    private UserService userService;

    @GetMapping("/{id}")
    public String getUser(@PathVariable Long id) {
        return userService.getUser(id);
    }
}
```

```
}  
}
```

启动类:

```
package com.example.demo;  
  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
@SpringBootApplication  
public class DemoApplication {  
  
    public static void main(String[] args) {  
        SpringApplication.run(DemoApplication.class, args);  
    }  
}
```

@Component是底层的通用注解

`@Component`，把一个普通类交给 Spring 容器管理（变成 Bean），是最底层通用注解

`@Component` 是 Spring IoC 容器“收编普通类”的入口注解

`@Service` / `@Repository` / `@Controller` 只是语义增强版的 `@Component`

Spring IOC/DI常用注解:

Bean注册类注解:

1. `@Component` : 最通用的底层注解,Bean注册注解
2. `@Service` : (业务层专用)层级注解,Bean注册注解
3. `@Repository` : (数据访问层专用)标识数据库访问类
4. `@Controller` : (Web控制层专用)处理 HTTP 请求,返回页面或数据

依赖注入注解(DI):

1. `@Autowired` : 自动注入依赖, Spring 自动帮你“找对象 + 注入”
2. `@Qualifier` : 指定注入哪个 Bean, 指定注入的 Bean 名称

请求处理相关(Web层):

1. `@RequestMapping` : 映射请求路径, 处理 HTTP 请求
2. `@GetMapping` / `@PostMapping` : 请求方法简化版
3. `@PathVariable` : 获取路径参数
4. `@RequestParam` : 获取请求参数
5. `@RequestBody` : 接收 JSON 请求体

Spring Boot 启动类:

1. `@SpringBootApplication` : 启动类注解, Spring Boot 启动类, 自动配置 Spring Boot 应用

IOC扫描核心:

1. @ComponentScan : 扫描组件

事务相关:

1. @Transactional : 事务注解,控制事务

- 自动开启事务
- 出错自动回滚

Mysql:关系型数据库

- Mysql初始化操作(在CMD上运行): `mysqld --initialize-insecure`
- 将Mysql安装为系统服务: `mysqld -install`
- 启动Mysql服务: `net start mysql`
- 想要停止Mysql服务: `net stop mysql`
- 修改Mysql默认的账户和密码: `mysqladmin -u root password 000000`

SQL连接: `mysql -u用户名 -p密码 [-h数据库服务器IP地址 -P端口号]`

SQL语句:

1. DDL:用来"定义数据结构"
2. DML:用来"操作数据(增删改)"
3. DQL:用来"查询数据"
4. DCL:用来"控制权限"

一、DDL (Data Definition Language) 数据定义语言:

作用:创建表、修改表、删除表(定义表的结构)

```
CREATE TABLE user (  
    id INT,  
    name VARCHAR(20)  
);  
  
ALTER TABLE user ADD age INT;  
  
DROP TABLE user;
```

DDL中常见的约束:

- `NOT NULL`(非空约束):非空
- `UNIQUE`(唯一约束):唯一
- `PRIMARY KEY`(主键约束):主键(唯一+非空)
- `FOREIGN KEY`(外键约束):外键(关联表,让两张表的数据连接,保证数据的一致性和完整性)
- `DEFAULT`(默认值约束):默认值(若未指定值,则采用默认值)
- `CHECK`(检查约束):条件限制
- `auto_increment`:主键自增,不用自己再添加

数据类型选取规则:在满足业务需求的前提下, 尽可能选择占用磁盘空间小的数据类型:

- `age tinyint unsigned` (0,255)范围, `unsigned` 指的是无符号的范围(但加上之后不能取负数)
- `id int unsigned`
- `varchar` 是动态的更改占用空间(节约磁盘空间,但性能略低,因为需要计算), 而`char`是写死的, 无论存没存都占空间(性能略高, 但磁盘空间利用率低)
- 大公司想要速度就用`char`, 小公司对性能没要求, 想节约磁盘空间就用`varchar`
-

常用操作:

```
--查询所有数据库
show databases;

--查询当前数据库
select databases;

--使用/切换数据库
use 数据库名;

--创建数据库
create database [if not exists] 数据库名 [default charset utf8mb4]

--删除数据库
drop database [if exists] 数据库名
```

除了前三个, 其他都是DDL

`database` 可以替换为 `schema` 提升打字效率

DDL主要操作数据结构(库、表、字段):

1. CREATE(创建):

创建数据库:

```
CREATE DATABASE mydb;
```

创建表:

```
CREATE TABLE user (
  id INT PRIMARY KEY,
  name VARCHAR(20),
  age INT
);
```

创建索引:

```
CREATE INDEX idx_name ON user(name);
```

DDL实战案例1 (表的创建, 根据页面原型, 设计员工表):

```
-- 案例：设计员工表
-- 基础字段：id 主键；create_time 创建时间；update_time 修改时间；
create table emp(
    id int unsigned primary key auto_increment comment 'ID, 主键',
    username varchar(20) not null unique comment '用户名',
    password varchar(32) default '000000' comment '密码',
    name varchar(10) not null comment '姓名',
    gender tinyint unsigned not null comment '性别, 1 男; 2 女',
    phone char(11) not null unique comment '手机号',
    job tinyint unsigned comment '职位, 1 班主任; 2 讲师; 3 学工主管; 4 教研主管; 5 咨询师',
    salary int unsigned comment '薪资',
    entry_date date comment '入职日期',
    image varchar(255) comment '图像',
    create_time datetime comment '创建时间',
    update_time datetime comment '修改时间'
) comment '员工表';
```

不要忘了添加基础字段(id 主键, create_time 创建时间, update_time 修改时间)

2. ALTER(修改):

添加字段:

```
ALTER TABLE user ADD email VARCHAR(50);
```

修改字段类型:

```
ALTER TABLE user MODIFY age BIGINT;
```

删除字段:

```
ALTER TABLE user DROP email;
```

3. DROP(删除):

删除表:

```
DROP TABLE user;
```

删除数据库:

```
DROP DATABASE mydb;
```

删除索引:

```
DROP INDEX idx_name ON user;
```

4. TRUNCATE(清空表):

清空表(删除快, 但不可回滚, 清空数据(结构还在))

```
TRUNCATE TABLE user;
```

5. RENAME(重命名):

修改表名:

```
RENAME TABLE user TO user_new;
```

DDL实战案例2 (表结构-查询、修改、删除) :

```
-- 查询当前数据库所有表
show tables;

-- 查看表结构
desc emp;

-- 查询建表语句
show create table emp;

-- 字段: 添加字段 qq varchar(13)
alter table emp add qq varchar(13) comment 'QQ号码';

-- 字段: 修改字段类型 qq varchar(15)
alter table emp modify qq varchar(15) comment 'QQ号码';

-- 字段: 修改字段名 qq -> qq_num varchar(15)
alter table emp change qq qq_num varchar(15) comment 'QQ号码';

-- 字段: 删除字段 qq_num
alter table emp drop column qq_num;

-- 修改表名
alter table emp rename to employee;

-- 删除表
drop table employee;
```

二、DML (Data Manipulation Language) 数据操作语言:

用来操作数据(增删改查)

```
INSERT INTO user VALUES (1, 'Tom');

UPDATE user SET name = 'Jerry' WHERE id = 1;

DELETE FROM user WHERE id = 1;
```

DML(操作数据库中的数据 (增、删、改)):

1. INSERT(插入数据,往表里新增数据):

```
INSERT INTO user (id, name, age)
VALUES (1, 'Tom', 18);
```

插入多条数据:

```
INSERT INTO user (id, name, age)
VALUES
(2, 'Jerry', 20),
(3, 'Mike', 22);
```

DML实战案例1(Insert):

```
-- DML : 插入数据 - insert
-- 1. 为 emp 表的 username, password, name, gender, phone 字段插入值
insert into emp(username,password,name,gender,phone) values ('songjiang','00000000','宋江','1','19384756473');

-- 2. 为 emp 表的所有字段插入值
-- 方式1:
insert into emp(id, username, password, name, gender, phone, job, salary, entry_date, image, create_time, update_time)
values(null,'linchong','00000000','林冲','1','10293485743',1,6000,'2020-01-01','1.jpg',now(),now())

-- 方式2:
insert into emp values(null,'likui','00000000','李逵','1','14593485743',1,6000,'2020-01-01','1.jpg',now(),now())

-- 3. 批量为 emp 表的 username, password, name, gender, phone 字段插入数据
insert into emp values
(null,'ruanxiaoer','00000000','阮小二','1','13893485743',1,6000,'2020-01-01','1.jpg',now(),now()),
(null,'ruanxiaowu','00000000','阮小五','1','16093485743',1,6000,'2020-01-01','1.jpg',now(),now())
```

2. UPDATE(更新数据,修改已有数据):

```
UPDATE user
SET name = 'Tony'
WHERE id = 1;
```

多字段更新:

```
UPDATE user
SET name = 'Lucy', age = 25
WHERE id = 2;
```

如果没有WHERE:(会更新整张表)

```
UPDATE user SET age = 100;
```

DML实战案例2(Update):

```
-- DML : 更新数据 - update
-- 1. 将 emp 表的ID为1员工 用户名更新为 'zhangsan', 姓名name字段更新为 '张三'
update emp set username = 'zhangsan' , name = '张三' where id = 1;
-- 上面就是 set 后跟更改后的数据, where是要更改哪个或哪些字段

-- 2. 将 emp 表的所有员工的入职日期更新为 '2010-01-01'
update emp set entry_date = '2019-01-01';
```

3. DELETE(删除数据,删除表中的数据):

删除指定数据:

```
DELETE FROM user
WHERE id = 1;
```

删除所有数据:

```
DELETE FROM user;
```

DML实战案例3(Delete):

```
-- DML : 删除数据 - delete
-- 1. 删除 emp 表中 ID为1的员工
delete from emp where id = 1;

-- 2. 删除 emp 表中的所有员工
delete from emp ;
```

三、DQL (Data Query Language) 数据查询语言:

用来查询数据

```
SELECT * FROM user;

SELECT name FROM user WHERE id = 1;
```

基本查询-数据准备:

```
-- DQL : 查询语句 数据准备
create table emp(
    id int unsigned primary key auto_increment comment 'ID,主键',
    username varchar(20) not null unique comment '用户名',
    password varchar(32) not null comment '密码',
    name varchar(10) not null comment '姓名',
    gender tinyint unsigned not null comment '性别, 1:男, 2:女',
    phone char(11) not null unique comment '手机号',
```

```
job tinyint unsigned comment '职位, 1:班主任,2:讲师,3:学工主管,4:教研主管,5:咨询师',
salary int unsigned comment '薪资',
image varchar(300) comment '头像',
entry_date date comment '入职日期',
create_time datetime comment '创建时间',
update_time datetime comment '修改时间'
) comment '员工表';

-- 准备测试数据
INSERT INTO emp(id, username, password, name, gender, phone, job, salary, image, entry_date,
create_time, update_time)
VALUES (1, 'shinaian', '123456', '施耐庵', 1, '13309090001', 4, 15000, '1.jpg', '2000-01-01', '2024-04-
11 16:35:33', '2024-04-11 16:35:35'),
(2, 'songjiang', '123456', '宋江', 1, '13309090002', 2, 8600, '2.jpg', '2015-01-01', '2024-04-11
16:35:33', '2024-04-11 16:35:37'),
(3, 'lujunyi', '123456', '卢俊义', 1, '13309090003', 2, 8900, '3.jpg', '2008-05-01', '2024-04-11
16:35:33', '2024-04-11 16:35:39'),
(4, 'wuyong', '123456', '吴用', 1, '13309090004', 2, 9200, '4.jpg', '2007-01-01', '2024-04-11
16:35:33', '2024-04-11 16:35:41'),
(5, 'gongsunsheng', '123456', '公孙胜', 1, '13309090005', 2, 9500, '5.jpg', '2012-12-05', '2024-
04-11 16:35:33', '2024-04-11 16:35:43'),
(6, 'huosanniang', '123456', '扈三娘', 2, '13309090006', 3, 6500, '6.jpg', '2013-09-05', '2024-04-
11 16:35:33', '2024-04-11 16:35:45'),
(7, 'chaijin', '123456', '柴进', 1, '13309090007', 1, 4700, '7.jpg', '2005-08-01', '2024-04-11
16:35:33', '2024-04-11 16:35:47'),
(8, 'likui', '123456', '李逵', 1, '13309090008', 1, 4800, '8.jpg', '2014-11-09', '2024-04-11
16:35:33', '2024-04-11 16:35:49'),
(9, 'wusong', '123456', '武松', 1, '13309090009', 1, 4900, '9.jpg', '2011-03-11', '2024-04-11
16:35:33', '2024-04-11 16:35:51'),
(10, 'lichong', '123456', '林冲', 1, '13309090010', 1, 5000, '10.jpg', '2013-09-05', '2024-04-11
16:35:33', '2024-04-11 16:35:53'),
(11, 'huyanzhuo', '123456', '呼延灼', 1, '13309090011', 2, 9700, '11.jpg', '2007-02-01', '2024-04-
11 16:35:33', '2024-04-11 16:35:55'),
(12, 'xiaoliguang', '123456', '小李广', 1, '13309090012', 2, 10000, '12.jpg', '2008-08-18', '2024-
04-11 16:35:33', '2024-04-11 16:35:57'),
(13, 'yangzhi', '123456', '杨志', 1, '13309090013', 1, 5300, '13.jpg', '2012-11-01', '2024-04-11
16:35:33', '2024-04-11 16:35:59'),
(14, 'shijin', '123456', '史进', 1, '13309090014', 2, 10600, '14.jpg', '2002-08-01', '2024-04-11
16:35:33', '2024-04-11 16:36:01'),
(15, 'sunerniang', '123456', '孙二娘', 2, '13309090015', 2, 10900, '15.jpg', '2011-05-01', '2024-
04-11 16:35:33', '2024-04-11 16:36:03'),
(16, 'luzhishen', '123456', '鲁智深', 1, '13309090016', 2, 9600, '16.jpg', '2010-01-01', '2024-04-
11 16:35:33', '2024-04-11 16:36:05'),
(17, 'liying', '12345678', '李应', 1, '13309090017', 1, 5800, '17.jpg', '2015-03-21', '2024-04-11
16:35:33', '2024-04-11 16:36:07'),
(18, 'shiqian', '123456', '时迁', 1, '13309090018', 2, 10200, '18.jpg', '2015-01-01', '2024-04-11
16:35:33', '2024-04-11 16:36:09'),
(19, 'gudasao', '123456', '顾大嫂', 2, '13309090019', 2, 10500, '19.jpg', '2008-01-01', '2024-04-
11 16:35:33', '2024-04-11 16:36:11'),
(20, 'ruanxiaoe', '123456', '阮小二', 1, '13309090020', 2, 10800, '20.jpg', '2018-01-01', '2024-
04-11 16:35:33', '2024-04-11 16:36:13'),
(21, 'ruanxiaowu', '123456', '阮小五', 1, '13309090021', 5, 5200, '21.jpg', '2015-01-01', '2024-
04-11 16:35:33', '2024-04-11 16:36:15'),
```

```
(22,'ruanxiaqi','123456','阮小七',1,'13309090022',5,5500,'22.jpg','2016-01-01','2024-04-11 16:35:33','2024-04-11 16:36:17'),
(23,'ruanji','123456','阮籍',1,'13309090023',5,5800,'23.jpg','2012-01-01','2024-04-11 16:35:33','2024-04-11 16:36:19'),
(24,'tongwei','123456','童威',1,'13309090024',5,5000,'24.jpg','2006-01-01','2024-04-11 16:35:33','2024-04-11 16:36:21'),
(25,'tongmeng','123456','童猛',1,'13309090025',5,4800,'25.jpg','2002-01-01','2024-04-11 16:35:33','2024-04-11 16:36:23'),
(26,'yanshun','123456','燕顺',1,'13309090026',5,5400,'26.jpg','2011-01-01','2024-04-11 16:35:33','2024-04-11 16:36:25'),
(27,'lijun','123456','李俊',1,'13309090027',5,6600,'27.jpg','2004-01-01','2024-04-11 16:35:33','2024-04-11 16:36:27'),
(28,'lizhong','123456','李忠',1,'13309090028',5,5000,'28.jpg','2007-01-01','2024-04-11 16:35:33','2024-04-11 16:36:29'),
(29,'songqing','123456','宋清',1,'13309090029',5,5100,'29.jpg','2020-01-01','2024-04-11 16:35:33','2024-04-11 16:36:31'),
(30,'liyun','123456','李云',1,'13309090030',NULL,NULL,'30.jpg','2020-03-01','2024-04-11 16:35:33','2024-04-11 16:36:31');
```

DQL:基本查询案例(基本查询):

```
-- ===== DQL: 基本查询 =====
-- 1. 查询指定字段 name,entry_date 并返回
select name,entry_date from emp;

-- 2. 查询返回所有字段
-- 方式1: 推荐(可以把要查询的字段罗列出来)
select id,username,id, username, password, name, gender, phone, job, salary, entry_date,
image, create_time, update_time from emp;

-- 方式2:不推荐
select * from emp;

-- 3. 查询所有员工的 name,entry_date, 并起别名(姓名、入职日期)
select name as 姓名,entry_date as 入职日期 from emp;

select name 姓名,entry_date 入职日期 from emp;

-- 4. 查询已有的员工关联了哪几种职位(不要重复) - distinct
select distinct job from emp;
```

1. WHERE(条件查询,过滤数据):

条件查询:

```
SELECT * FROM user
WHERE age > 18;
```

多条件查询:


```
SELECT * FROM user
WHERE age > 18 AND name = 'Tom';
```

AND连接前后

DQL:案例1(条件查询)

```
-- ===== DQL: 条件查询 =====
-- 1. 查询 姓名 为 柴进 的员工
select * from emp where name = '柴进';

-- 2. 查询 薪资小于等于5000 的员工信息
select * from emp where salary <= 5000;

-- 3. 查询 没有分配职位 的员工信息
select * from emp where job is null;

-- 4. 查询 有职位 的员工信息
select * from emp where job is not null;

-- 5. 查询 密码不等于 '123456' 的员工信息
select * from emp where password != '123456';
-- 下面这种方式不推荐, 如果别人这样写你能知道就行
select * from emp where password <> '123456';

-- 6. 查询 入职日期 在 '2000-01-01' (包含) 到 '2010-01-01'(包含) 之间的员工信息
select * from emp where entry_date between '2000-01-01' and '2010-01-01';

-- 7. 查询 入职时间 在 '2000-01-01' (包含) 到 '2010-01-01'(包含) 之间 且 性别为女 的员工信息
select * from emp where entry_date between '2000-01-01' and '2010-01-01' and gender = 2;

-- 8. 查询 职位是 2 (讲师), 3 (学工主管), 4 (教研主管) 的员工信息
select * from emp where job = 2 or job = 3 or job = 4;
select * from emp where job in(2,3,4);
-- 两种方式都可以

-- 9. 查询 姓名 为两个字的员工信息( _ :单个字符; % 任意个字符)
select * from emp where name like '__';

-- 10. 查询 姓 '李' 的员工信息
select * from emp where name like '李%';

-- 11. 查询 姓名中包含 '二' 的员工信息
select * from emp where name like '%二%';
```

2. ORDER BY(排序):

升序排序:

```
SELECT * FROM user
ORDER BY age ASC;
```

ASC 表示升序

降序排序:

```
SELECT * FROM user
ORDER BY age DESC;
```

DESC 表示降序

DQL:案例2(排序查询)

```
-- ===== 排序查询 =====
-- 1. 根据入职时间，对员工进行升序排序 - asc
select * from emp order by entry_date asc;
select * from emp order by entry_date;

-- 2. 根据入职时间，对员工进行降序排序 - desc
select * from emp order by entry_date desc;

-- 3. 根据 入职时间 对公司的员工进行 升序排序 ， 入职时间相同 ， 再按照 更新时间 进行降序排序
select * from emp order by entry_date,update_time desc;
```

3. LIMIT(分页):

限制查询条数:

```
SELECT * FROM user
LIMIT 5;
```

分页查询:

```
SELECT * FROM user
LIMIT 0, 10;
```

DQL:案例3(分页查询)

```
-- ===== 分页查询 =====
-- 1. 从起始索引0开始查询员工数据，每页展示5条记录
-- limit [起始索引],[展示的记录数]
select * from emp limit 0,5;
select * from emp limit 10;

-- 2. 查询 第1页 员工数据，每页展示5条记录
select * from emp limit 0,5;

-- 3. 查询 第2页 员工数据，每页展示5条记录
select * from emp limit 5,5;

-- 4. 查询 第3页 员工数据，每页展示5条记录
select * from emp limit 10,5;
```

```
-- 页码
-- 起始索引
```

4. GROUP BY(分组):

按字段分组:

```
SELECT age, COUNT(*)
FROM user
GROUP BY age;
```

搭配聚合函数:

- COUNT (计数)
- SUM (求和)
- AVG (平均)
- MAX (最大值)
- MIN (最小值)

DQL:案例4(聚合函数)

```
-- ===== DQL: 分组查询 =====
-- 聚合函数
-- 注意: 所有的聚合函数不参与null的统计
-- 1. 统计该企业员工数量 - count
-- count(字段)
select count(id) from emp;

-- count(*) : 推荐
select count(*) from emp;

-- count(常量) : 推荐
select count(1) from emp;

-- 2. 统计该企业员工的平均薪资
select avg(salary) from emp;

-- 3. 统计该企业员工的最低薪资
select min(salary) from emp;

-- 4. 统计该企业员工的最高薪资
select max(salary) from emp;

-- 5. 统计该企业每月要给员工发放的薪资总额(薪资之和)
select sum(salary) from emp;
```

5. HAVING(分组后过滤):

对分组结果过滤:

```
SELECT age, COUNT(*)
FROM user
GROUP BY age
HAVING COUNT(*) > 1;
```

WHERE 分组前过滤,不能用聚合函数

HAVING 分组后过滤,可以使用聚合函数

where与having的区别:

- 执行时机不同,where是分组之前过滤(不满足where条件,不参与分组),having是分组之后对结果进行过滤
- 判断条件不同,where不能对聚合函数进行判断,having可以

DQL:案例5(分组查询)

```
-- 分组
-- 注意: 分组之后, select后的字段列表不能随意书写, 能写的一般是分组字段 + 聚合函数
-- 1. 根据性别分组, 统计男性和女性员工的数量
select gender, count(*) from emp group by gender;

-- 2. 先查询入职时间在 '2015-01-01' (包含) 以前的员工, 并对结果根据职位分组, 获取员工数量大于等于2的职位
select job, count(*) from emp where entry_date <= '2015-01-01' group by job having count(*)
>= 2;

-- 上面where在分组之前进行过滤, 之后用having在分组之后进行过滤
```

6. JOIN(多表查询,重点中的重点):

INNER JOIN(内连接):

```
SELECT u.name, o.order_id
FROM user u
INNER JOIN orders o ON u.id = o.user_id;
```

LEFT JOIN(左连接):

```
SELECT u.name, o.order_id
FROM user u
LEFT JOIN orders o ON u.id = o.user_id;
```

DQL总结:

SELECT	→ 查询数据
WHERE	→ 条件过滤
ORDER BY	→ 排序
LIMIT	→ 分页
GROUP BY	→ 分组
HAVING	→ 分组过滤
JOIN	→ 多表查询

执行顺序的优先级:

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

SELECT → 查东西
WHERE → 挑选条件
ORDER BY → 排队排序
LIMIT → 只拿一部分
GROUP BY → 分组统计

四、DCL (Data Control Language) 数据控制语言:

用来控制权限

```
GRANT SELECT ON user TO 'tom';  
  
REVOKE SELECT ON user FROM 'tom';
```

DDL: 定义结构 (create / alter / drop)
DML: 操作数据 (insert / update / delete)
DQL: 查询数据 (select)
DCL: 权限控制 (grant / revoke)

DataGrip中的快捷键:

Alt + 1 左边侧边栏隐藏或开启
Alt + 2 右边侧边栏隐藏或开启
Ctrl + Enter 运行SQL语句
IDEA中也有类似的,需要自行探索

JDBC:Java自带的连接数据库的一个库(工具):

- 静态SQL,用的是 Statement
- 动态SQL,用的是 PreparedStatement
- JDBC步骤:驱动连接--> 创建PreparedStatement(推荐)--> 执行SQL--> 处理结果--> 关闭连接

预编译SQL:

- 可以防止SQL注入攻击
- 占位符: ?
- 性能更高(因为第一次查找过后就会缓存SQL,下次执行SQL的时候就不需要再次查找SQL了)

SQL注入:

- 登录本质上就是查询,注入SQL语句: ' or '1'='1 就是 或者 1=1, 1肯定是恒等于1, 所以就会显示登录成功
- 账号随便输, 密码就是上面SQL语句中的 ' or '1'='1, 登录成功,所以需要预编译SQL来防止SQL注入攻击

JDBC:Java的一个底层连接数据库的库(工具)

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;

public class JdbcTest {
    public static void main(String[] args) throws Exception {

        //1. 注册驱动
        Class.forName("com.mysql.cj.jdbc.Driver");

        //2. 获取数据库连接
        String url = "jdbc:mysql://localhost:3306/web01";
        String username = "root";
        String password = "000000";
        Connection connection = DriverManager.getConnection(url, username, password);

        // 3. 获取SQL语句执行对象(给连接封装一下)
        Statement statement = connection.createStatement();

        // 4. 执行SQL语句
        int i = statement.executeUpdate("update user set age = 25 where id = 1");
        System.out.println("SQL执行完毕影响的记录数为:"+i);

        //5. 释放资源
        statement.close();
        connection.close();
    }
}

```

一般不适用JDBC，而是使用MyBatis，JDBC有点繁琐
不过MyBatis还有一些其他的三方库都是基于JDBC封装的

MyBatis:是一款优秀的持久层，用于简化JDBC的开发

- controller(控制层) ---> service(业务层) ---> dao(持久层)
- dao(持久层)

MyBatis:Mapper接口

```

import java.util.List;

@Mapper
public interface UserMapper {
    /**
     * 查询全部
     */
    @Select("select * from user")
    public List<User> findAll();
}

```

IDEA没有识别你的SpringBoot项目怎么办?

- 找到你SpringBoot项目里的pom.xml文件，右键，选择 Add as Maven Project

- 之后重新加载一下你的Maven，确保你的SpringBoot项目被识别了

Mybatis核心组件分工:

组件	作用	
-----	-----	
Mapper接口	定义方法	
Mapper.xml	写SQL	
SqlSession	执行SQL	
Executor	真正执行	
ResultMap	结果映射	
Parameter	参数绑定	
Configuration	全局配置	

SpringBoot使用Mybatis前置配置:

打开 `resources` 目录下SpringBoot的配置文件:

application.properties

```
# 配置数据库的连接
spring.datasource.url=jdbc:mysql://localhost:3306/web01
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.username=root
spring.datasource.password=000000
```

UserMapper.java

```
package com.shadow.mapper;

import com.shadow.pojo.User;
import org.apache.ibatis.annotations.Mapper;
import org.apache.ibatis.annotations.Select;

import java.util.List;

// 应用程序在运行时,会自动的为该接口创建一个实现类对象(代理对象)
// @Mapper // 并且会自动将该实现类对象存入IOC容器中
public interface UserMapper {
    /**
     * 查询所有用户信息
     */
    @Select("select * from user")
    public List<User> findAll();
}
```

Mapper接口 = “你要干啥（方法声明）”

MyBatis是外卖平台，那Mapper接口 = 点菜按钮

编写测试类:

```
package com.shadow;
```

```

import com.shadow.mapper.UserMapper;
import com.shadow.pojo.User;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;

import java.util.List;

@SpringBootTest // SpringBoot单元测试的注解 - 当前测试类中的测试方法运行时，会启动springboot项目 - IOC容器
class SpringbootMybatis1ApplicationTests {

    @Autowired
    private UserMapper userMapper;

    @Test
    void contextLoads() {
        List<User> userList = userMapper.findAll();
        userList.forEach(System.out::println);
    }

}

```

总结:

1.SpringBoot+Mybatis入门程序操作步骤:

准备工作:

- 创建一个SpringBoot项目，并引入MyBatis依赖
- 准备数据库、实体类
- `application.properties` 中配置数据库连接信息

定义Mapper接口(`@Mapper`)，编写SQL

2.SpringBoot的单元测试类上加注解

- `@SpringBootTest`:会在单元测试时，加载SpringBoot的环境
- 注意:测试类所在的包名必须和主类在同一个包下，或者子包下

Mybatis的配置:

`application.properties`

```

#配置mybatis的日志输出
mybatis.configuration.log-impl=org.apache.ibatis.logging.stdout.StdOutImpl

```

数据库连接池:

- SpringBoot自带的连接池,默认是HikariCP,追光者连接池
- Druid(德鲁伊)连接池,是阿里巴巴开源的数据库连接池项目

切换数据库连接池:

pom.xml

```
<!--Druid连接池-->
<dependency>
    <groupId>com.alibaba</groupId>
    <artifactId>druid-spring-boot-starter</artifactId>
    <version>1.2.19</version>
</dependency>
```

application.properties

```
spring.datasource.type=com.alibaba.druid.pool.DruidDataSource
```

Mybatis-增删改查-1.删除操作:

UserMapper.java

```
/**
 * 根据ID删除用户信息
 */
@Delete("delete from user where id = #{id}")
public void deleteById(Integer id);
```

SpringbootMybatis1ApplicationTests.java

```
/**
 * 测试删除用户
 */
public void testDeleteById() {
    userMapper.deleteById(5);
}
```

如果想要看返回值:

UserMapper.java

```
/**
 * 根据ID删除用户信息
 */
@Delete("delete from user where id = #{id}")
public Integer deleteById(Integer id);
```

把返回值改为Integer

SpringbootMybatis1ApplicationTests.java

```
/**
 * 测试删除用户
 */
Integer i = userMapper.deleteById(5);
System.out.println("执行完毕，影响的记录数:"+i);
```

输出结果为:执行完毕,影响的记录数:1

Mybatis中的 # 号与 \$ 号:

号表示占位符,生成预编译SQL(安全,性能高)

\$号表示变量,拼接符参数直接拼接(不安全),表明、字段名动态设置时使用

\$号直接传变量,还是会存在SQL注入的问题, # 占位符,不会被SQL注入

Mybatis-增删改查-2.新增操作:

UserMapper.java

```
/**
 * 添加用户信息
 */
@Insert("insert into user(username,password,name,age) values("#{username}","#{password}","#{name}","#{age})")
public void insert(User user);
```

SpringbootMybatis1ApplicationTests.java

```
/**
 * 测试插入用户
 */
@Test
public void testInsert(){
    User user = new User(null,"gaoyuanyuan","666888","高圆圆",18);
    userMapper.insert(user);
}
```

Mybatis-增删改查-3.修改操作:

UserMapper.java

```
/**
 * 修改用户信息
 */
@Update("update user set username = #{username},password=#{password},name=#{name},age = #{age} where id = #{id}")
public void update(User user);
```

SpringbootMybatis1ApplicationTests.java

```
/**
 * 测试更新用户
 */
@Test
public void testUpdate(){
    User user = new User(1,"zhouyu","666888","周瑜",20);
    userMapper.update(user);
}
```

把需要更改的值封装到对象中,然后调用update方法

Mybatis-增删改查-4.查询操作:

UserMapper.java

```
/**
 * 查询用户信息
 * @Param 注解的作用是为接口的方法形参起名字的(当你要传递多个参数的时候)
 */
@Select("select * from user where username = #{username} and password = #{password}")
public User findByUsernameAndPassword(@Param("username") String username,
@Param("password") String password);
```

@Param 就是给形参起名字的(当你要传递多个参数的时候)

基于官方骨架创建的springboot项目中,接口编译时,会保留方法形参名,@Param注解可以省略("#{形参名})

SpringbootMybatis1ApplicationTests.java

```
/**
 * 根据用户名和密码查询用户信息
 */
@Test
public void testFindByUsernameAndPassword(){
    User user = userMapper.findByUsernameAndPassword("zhouyu", "666888");
    System.out.println(userMapper);
}
```

报错从下往上找

XML映射配置:

- 在Mybatis中,既可以通过注解配置SQL语句,也可以通过XML配置SQL语句
- 默认规则:
 1. XML映射文件的名称与Mapper接口名称一致,并且需要放到相同包下(同包同名)
 2. XML映射文件的 namespace 属性为Mapper接口全限定名一致(namespace 与Mapper接口名相一致)
 3. XML映射文件中sql语句的id与Mapper 接口中的方法名一致,并保持返回类型一致(id 右边是方法名)

UserMapper.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="com.shadow.mapper.UserMapper">
    <!-- resultType: 查询返回的单条记录所封装的类型 -->
    <select id="findAll" resultType="com.shadow.pojo.User">
        select id, username, password, name, age from user
    </select>
</mapper>
```

到底使用注解开发还是使用XML开发呢?

1. 开发简单(简单的CRUD), 使用注解开发
2. 如果可以实现复杂的SQL功能,建议使用XML来配置映射语句

XML映射文件-辅助配置:

指定XML映射配置文件的位置:

application.properties

```
mybatis.mapper-locations=classpath:mapper/*.xml
```

IDEA插件: Mybatisx

SpringBoot项目配置文件:

```
spring.application.name=springboot-mybatis1

# 配置数据库的连接
spring.datasource.type=com.alibaba.druid.pool.DruidDataSource
spring.datasource.url=jdbc:mysql://localhost:3306/web01
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.username=root
spring.datasource.password=000000

#配置mybatis的日志输出
mybatis.configuration.log-impl=org.apache.ibatis.logging.stdout.StdOutImpl

#指定XML映射配置文件的位置
mybatis.mapper-locations=classpath:mapper/*.xml
```

```
application.properties
```

配置文件格式:

- SpringBoot 默认支持 `application.properties` 和 `application.yml` 等格式。
- 详细查看配置文件的书写格式。

application.yaml

```
# 定义对象/Map集合
user:
  name: "Tome"
  age: 18
  gender: 男

# 定义数组/List/Set集合
hobby:
  - Java
  - Game
  - Sport
```

演示1

application.yaml

```
# 项目的名称
spring:
  application:
    name: springboot-mybatis1
# 数据库的连接信息
datasource:
  type: com.alibaba.druid.pool.DruidDataSource
  url: jdbc:mysql://localhost:3306/web01
  driver-class-name: com.mysql.cj.jdbc.Driver
  username: root
  password: 000000

# Mybatis的相关配置
mybatis:
  configuration:
    log-impl: org.apache.ibatis.logging.stdout.StdOutImpl
  mapper-locations: classpath:mapper/*.xml
```

Spring项目需要的配置:

- `UserMapper.xml` 定义Mapper接口的SQL语句(数据库的SQL语句)
- `application.yaml` 定义SpringBoot项目所需的配置

Ajax 用于接口API,而 websocket 常用于实时通信(微信,QQ)

前端

├─ AJAX → REST API (业务操作)
└─ websocket → 实时推送

Tlias智能学习辅助系统,后端开发:

接口文档:

- 接口文档是对后端 API 接口的说明文档,用于描述接口如何调用、传什么参数、返回什么结果
- 避免了前端不知道接口怎么调,后端不知道前端要什么
- 而接口文档是需要基于页面原型+需求去制作的

接口文档是对系统 API 的规范化说明,包含接口地址、请求方式、参数及返回结果等信息,用于指导前后端开发与系统集成。

前后端开发流程:

- 需求分析 ---> 接口设计(API接口文档) ---> 前后端并行开发(遵守规范) ---> 测试(前端、后端) ---> 前后端联调测试

Restful(REpresentational State Transfer):

- 表述性状态转换,它是一种软件架构风格
- RESTful 是一种基于 HTTP 协议的 API 设计风格,用来规范“如何设计接口”

REST中请求方式代表操作:

- GET 代表查询操作
- DELETE 代表删除操作
- POST 新增用户
- PUT 修改用户

接口测试软件:

- Postman 和 Apifox 都可以进行接口测试,一般使用 Apifox , 因为 Apifox 功能强大

为什么要使用接口测试软件?

- 因为如果我们要进行接口的测试,而浏览器默认是 GET 请求,而我们需要手写代码去测试
- 非常繁琐,所以使用接口测试软件

Tlias工程搭建:

- 创建 SpringBoot 工程,并引入web开发起步依赖、mybatis、mysql驱动、lombok。
- 创建数据库 dept,并在 application.yml 中配置数据库的基本信息
- 准备基础代码结构,并引入实体类 Dept 及统一的响应结果封装类 Result

在创建好的项目中 java.com.xxx 目录下:

- 创建 controller 包,并创建 DeptController 类,并添加 @RestController 注解
- 创建 service 包(业务逻辑),并创建 impl.DeptServiceImpl 类,并添加 @Service 注解.创建 DepService 接口
- 创建 mapper 包(映射类存放),并创建 DeptMapper 类,并添加 @Mapper 注解
- 创建 pojo 包(实现类存放),并创建 Dept 类和 Result 类

查询部门 (功能开发) :

接口开发:

- 根据 页面原型 + API文档 进行接口开发
- Controller:接收请求 ---> Service:业务逻辑,调用 Mapper 接口 ---> Mapper :处理数据的访问

DeptController.java

```
package com.shadow.controller;

import com.shadow.pojo.Dept;
import com.shadow.pojo.Result;
import com.shadow.service.DeptService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;
```

```

@RestController
public class DepController {

    @Autowired
    private DeptService deptService;
    //@RequestMapping(value = "/depts",method = RequestMethod.GET)
    @GetMapping("/depts")
    public Result list(){
        System.out.println("查询全部部门数据");
        List<Dept> deptList = deptService.findAll();
        return Result.success(deptList);
    }
}

```

`Controller` 层接收数据，并调用 `Service` 层进行业务逻辑处理，并返回结果给前端

DeptServiceImpl.java

```

package com.shadow.service.impl;

import com.shadow.mapper.DeptMapper;
import com.shadow.pojo.Dept;
import com.shadow.service.DeptService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class DeptServiceImpl implements DeptService {

    @Autowired
    private DeptMapper deptMapper;
    @Override
    public List<Dept> findAll() {
        return deptMapper.findAll();
    }
}

```

`Service` 层进行业务逻辑处理，并调用 `Mapper` 层进行数据访问

DeptMapper.java

```

package com.shadow.mapper;

import com.shadow.pojo.Dept;
import org.apache.ibatis.annotations.Mapper;
import org.apache.ibatis.annotations.Select;

import java.util.List;

@Mapper
public interface DeptMapper {

```

```

/**
 * 查询所有的部门数据
 */
@Select("select id, name, create_time, update_time from dept order by update_time desc")
List<Dept> findAll();
}

```

Mapper 层进行数据访问，并返回结果给 Service 层

Mybatis结果封装:

实体类属性名 和 数据库表查询返回的字段名一致,mybatis会自动封装

如果实体类属性名 和 数据库表查询返回的字段名不一致,不能自动封装

解决方法:

1.使用 @Results 注解指定封装规则

2.在SQL语句中为字段添加别名

3.开启驼峰命名自动映射(推荐),在 application.yaml 中进行配置

1. @Results 手动结果映射:

- 通过 @Results 及 @Result 注解,进行手动结果映射

DeptMapper.java

```

/**
 * 查询所有的部门数据
 */
@Results({
    @Result(column = "create_time" , property = "createTime"),
    @Result(column = "update_time" , property = "updateTime")
})
@Select("select id, name, create_time, update_time from dept order by update_time desc")
List<Dept> findAll();

```

左边的 column 是数据库的字段名，右边的 property 是实体类的属性名

手动对结果进行映射

2.在SQL语句中为字段添加别名:

```

@Select("select id as deptId, name as deptName, create_time createTime, update_time updateTime from dept order by update_time desc")
List<Dept> findAll();

```

3.开启驼峰命名自动映射(推荐):


```
# Mybatis的相关配置
mybatis:
  configuration:
    log-impl: org.apache.ibatis.logging.stdout.StdOutImpl
    # 开启驼峰命名映射开关
    map-underscore-to-camel-case: true
```

前后端联调测试:

Nginx的反向代理:

Nginx/conf/nginx.conf

```
server {
    listen 90;
    #省略...
    location ^~ /api/ {
        rewrite ^/api/(.*)$ /$1 break;
        proxy_pass http://localhost:8080;
    }
}
```

- `location`: 用于定义匹配路径匹配的规则,比如上面就是配置的正则
- `^~ /api/`:表示精确匹配,即只匹配以/api/开头的请求
- `rewrite`: 用于将匹配到的请求路径进行重写,比如上面就是将 `/api/xxx` 重写为 `/xxx`
- `proxy_pass`: 用于代理转发,将匹配到的请求发给位于后端的指令服务器

这段代码主要是把 `/api/xxx` 后面的路径给截下来,然后转发给 `http://localhost:8080/xxx`

删除部门(功能开发):

参数传递:

方式一: `HttpServletRequest` 获取请求参数

```
/**
 * 删除部门 - 方式一: HttpServletRequest 获取请求参数
 */
@DeleteMapping("/depts")
public Result delete(HttpServletRequest request){
    String idStr = request.getParameter("id");
    int id = Integer.parseInt(idStr);
    System.out.println("删除部门id为: " + id);
    return Result.success();
}
```

方式二:@RequestParam 获取请求参数

```

/**
 * 删除部门 - 方式二: @RequestParam 获取请求参数
 */
@DeleteMapping("/depts")
public Result delete(@RequestParam("id") Integer deptId){
    System.out.println("删除部门id为: " + deptId);
    return Result.success();
}

```

@RequestParam 注解用于获取请求参数

```

/**
 * 删除部门 - 方式二: @RequestParam 获取请求参数
 * 注意事项: 一旦声明了@RequestParam注解, 该参数在请求时必须传递, 如果不传递将会报错(默认 required
为 true)
 */
@DeleteMapping("/depts")
public Result delete(@RequestParam(value = "id",required = false) Integer deptId) {
    System.out.println("删除部门id为: " + deptId);
    return Result.success();
}

```

@RequestParam 注解的 required 属性默认为 true,表示该参数必须传递,如果不传递将会报错
如果设置为false则可以为空

方式三:省略@RequestParam(前端传递的请求参数与服务端方法形参名一致)(推荐)

```

/**
 * 删除部门 - 方式三: 省略@RequestParam(前端传递的请求参数与服务端方法形参名一致)
 */
@DeleteMapping("/depts")
public Result delete(Integer deptId) {
    System.out.println("删除部门id为: " + deptId);
    return Result.success();
}

```

- 如果你请求的是 ?id=1,那么服务端方法中的参数名就是 id
- 如果你请求的是 ?deptId=1,那么服务端方法中的参数名就是 deptId
- 前端传递的请求参数与服务端方法形参名一致

删除部门(功能实现):

DepController.java

```

/**
 * 删除部门 - 方式三：省略@RequestParam(前端传递的请求参数与服务端方法形参名一致)
 */
@DeleteMapping("/depts")
public Result delete(Integer id) {
    System.out.println("删除部门id为: " + id);
    deptService.deleteById(id);
    return Result.success();
}

```

Controller 层进行数据访问,并返回结果给 Service 层

DeptServiceImpl.java

```

@Override
public void deleteById(Integer id) {
    deptMapper.deleteById(id);
}

```

Service 层进行业务逻辑处理

DeptMapper.java

```

/**
 * 根据id删除部门
 */
@Delete("delete from dept where id = #{id}")
void deleteById(Integer id);

```

Mapper 层进行数据访问

整体逻辑: Controller (接收请求和返回数据) ---> Service (业务逻辑) ---> Mapper (数据访问)

新增部门(功能开发):

DepController.java

```

/**
 * 新增部门
 */
@PostMapping("/depts")
public Result add(@RequestBody Dept dept){
    System.out.println("新增部门: " + dept);
    deptService.add(dept);
    return Result.success();
}

```

@RequestBody 注解用于获取请求体中的数据

dept 是前端传递的请求参数

DeptServiceImpl.java

```

@Override
public void add(Department dept) {
    // 1. 补充基础属性 - createTime , updateTime
    dept.setCreateTime(LocalDateTime.now());
    dept.setUpdateTime(LocalDateTime.now());
    // 2. 调用Mapper接口方法插入数据
    deptMapper.insert(dept);
}

```

需要补充基础属性 - createTime , updateTime

DeptMapper.java

```

/**
 * 新增部门
 */
@Insert("insert into dept(name,create_time,update_time) values (#{name},#{createTime},#{updateTime})")
void insert(Department dept);

```

Mapper 层进行数据访问

修改部门(功能开发):

1.查询回显(根据id查询部门):

DepController.java

```

/**
 * 根据ID查询部门
 */
@GetMapping("/depts/{id}")
public Result getInfo(@PathVariable("id") Integer deptId){
    System.out.println("根据ID查询部门 : "+ deptId);
    return Result.success();
}

```

简化写法:

```

/**
 * 根据ID查询部门
 */
@GetMapping("/depts/{id}")
public Result getInfo(@PathVariable Integer id){
    System.out.println("根据ID查询部门 : "+ id);
    Dept dept = deptService.getById(id);
    return Result.success(dept);
}

```

如果路径参数与方法形参名一致,那么可以省略@PathVariable注解

DeptServiceImpl.java

```
@Override
public Dept getById(Integer id) {
    return deptMapper.getById(id);
}
```

DeptMapper.java

```
/**
 * 根据id查询部门
 */
@Select("select id,name,create_time,update_time from dept where id = #{id}")
Dept getById(Integer id);
```

- 为什么需要查询回显?
- 如果不回显:用户要改部门名称 → 但不知道当前叫什么
- 如果回显:用户要改部门名称 → 拿到当前部门的名称,就可以进行修改
- 回显后的流程:数据库数据 → 查询 → 展示给用户 → 用户修改 → 提交

2.修改数据:

DepController.java

```
/**
 * 修改部门
 */
@PutMapping("/depts")
public Result update(@RequestBody Dept dept){
    System.out.println("修改部门: " + dept);
    deptService.update(dept);
    return Result.success();
}
```

`@RequestBody`: 用于获取请求体中的数据

DeptServiceImpl.java

```
@Override
public void update(Dept dept) {
    // 1. 补全基础属性-updateTime
    dept.setUpdateTime(LocalDateTime.now());

    // 2. 调用Mapper接口方法更新部门
    deptMapper.update(dept);
}
```

- 补全基础属性- updateTime
- 调用 Mapper 接口方法更新部门

DeptMapper.java

```

/**
 * 更新部门
 */
@update("update dept set name = #{name} , update_time = #{updateTime} where id = #
{id};")
void update(Department dept);

```

更新部门的SQL语句，使用了MyBatis的注解方式

公共路径抽取(@RequestMapping):

DepController.java

```

@RequestMapping("/depts")
@RestController
public class DepController {

    @Autowired
    private DeptService deptService;
    // @RequestMapping(value = "/depts", method = RequestMethod.GET)
    @GetMapping
    public Result list(){
        System.out.println("查询全部部门数据");
        List<Dept> deptList = deptService.findAll();
        return Result.success(deptList);
    }

    /**
     * 删除部门 - 方式一: HttpServletRequest 获取请求参数
     */
    // @DeleteMapping("/depts")
    // public Result delete(HttpServletRequest request){
    //     String idStr = request.getParameter("id");
    //     int id = Integer.parseInt(idStr);
    //     System.out.println("删除部门id为: " + id);
    //     return Result.success();
    // }

    /**
     * 删除部门 - 方式二: @RequestParam 获取请求参数
     * 注意事项: 一旦声明了@RequestParam注解, 该参数在请求时必须传递, 如果不传递将会报错(默认 required
    为 true)
     */
    // @DeleteMapping("/depts")
    // public Result delete(@RequestParam(value = "id", required = false) Integer deptId){
    //     System.out.println("删除部门id为: " + deptId);
    //     return Result.success();
    // }

    /**
     * 删除部门 - 方式三: 省略@RequestParam(前端传递的请求参数与服务端方法形参名一致)
     */
    @DeleteMapping
    public Result delete(Integer id) {

```

```

        System.out.println("删除部门id为: " + id);
        deptService.deleteById(id);
        return Result.success();
    }

    /**
     * 新增部门
     */
    @PostMapping
    public Result add(@RequestBody Dept dept){
        System.out.println("新增部门: " + dept);
        deptService.add(dept);
        return Result.success();
    }

    /**
     * 根据ID查询部门
     */
    @GetMapping("/depts/{id}")
    public Result getInfo(@PathVariable("id") Integer deptId){
        System.out.println("根据ID查询部门 : "+ deptId);
        return Result.success();
    }

    /**
     * 根据ID查询部门
     */
    @GetMapping("/{id}")
    public Result getInfo(@PathVariable Integer id){
        System.out.println("根据ID查询部门 : "+ id);
        Dept dept = deptService.getById(id);
        return Result.success(dept);
    }

    /**
     * 修改部门
     */
    @PutMapping
    public Result update(@RequestBody Dept dept){
        System.out.println("修改部门: " + dept);
        deptService.update(dept);
        return Result.success();
    }
}

```

直接抽取 @RequestMapping("/depts")

Logback日志技术:

- **Logback:** 是SpringBoot默认的日志技术
- **Logback:** 基于**Log4j**升级而来,提供了更多的功能和配置选项,性能优于Log4j
- **Slf4j:** 是 **Logback** 的抽象层,提供了统一的日志接口,任何日志实现都可以实现这个接口,但是底层实现是

Logback 或 Log4j

在 resources 目录下创建 logback.xml 配置文件:

logback.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
    <!-- 控制台输出 -->
    <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
        <encoder class="ch.qos.logback.classic.encoder.PatternLayoutEncoder">
            <!--格式化输出: %d表示日期, %thread表示线程名, %-5level: 级别从左显示5个字符宽度
            %logger{50}: 最长50个字符(超出.切割) %msg: 日志消息, %n是换行符 -->
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{50} -
            %msg%n</pattern>
        </encoder>
    </appender>

    <!-- 日志输出级别 -->
    <root level="debug">
        <appender-ref ref="STDOUT" />
    </root>
</configuration>
```

入门程序:

LogTest.xml:

```
package com.shadow;

import org.junit.jupiter.api.Test;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.time.LocalDateTime;

public class LogTest {

    // 日志记录器
    private static final Logger log = LoggerFactory.getLogger(LogTest.class);
    @Test
    public void testLog(){
        //System.out.println(LocalDateTime.now() + " : 开始计算...");
        log.debug("开始计算...");

        int sum = 0;
        int[] nums = {1, 5, 3, 2, 1, 4, 5, 4, 6, 7, 4, 34, 2, 23};
        for (int num : nums) {
            sum += num;
        }

        log.info("计算结果为: "+ sum);
        //System.out.println("计算结果为: "+sum);
    }
}
```



```

        //System.out.println(LocalDateTime.now() + "结束计算...");
        log.debug("结束计算...");
    }

}

```

运行结果:

```

2026-05-04 14:19:34.574 [main] DEBUG com.shadow.LogTest - 开始计算...
2026-05-04 14:19:34.577 [main] INFO com.shadow.LogTest - 计算结果为: 101
2026-05-04 14:19:34.578 [main] DEBUG com.shadow.LogTest - 结束计算...

```

完整的 LogTest.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<configuration>
    <!-- 控制台输出 -->
    <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
        <encoder class="ch.qos.logback.classic.encoder.PatternLayoutEncoder">
            <!--格式化输出: %d 表示日期, %thread 表示线程名, %-5level表示级别从左显示5个字符宽度,
%logger显示日志记录器的名称, %msg表示日志消息, %n表示换行符 -->
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{50}-
%msg%n</pattern>
        </encoder>
    </appender>

    <!-- 系统文件输出 -->
    <appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
        <rollingPolicy class="ch.qos.logback.core.rolling.SizeAndTimeBasedRollingPolicy">
            <!-- 日志文件输出的文件名, %i表示序号 -->
            <fileNamePattern>D:/tlias-%d{yyyy-MM-dd}-%i.log</fileNamePattern>
            <!-- 最多保留的历史日志文件数量 -->
            <maxHistory>30</maxHistory>
            <!-- 最大文件大小, 超过这个大小会触发滚动到新文件, 默认为 10MB -->
            <maxFileSize>10MB</maxFileSize>
        </rollingPolicy>

        <encoder class="ch.qos.logback.classic.encoder.PatternLayoutEncoder">
            <!--格式化输出: %d 表示日期, %thread 表示线程名, %-5level表示级别从左显示5个字符宽度, %msg
表示日志消息, %n表示换行符 -->
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{50}-
%msg%n</pattern>
        </encoder>
    </appender>

    <!-- 日志输出级别 -->
    <root level="ALL">
        <appender-ref ref="STDOUT" />
        <appender-ref ref="FILE" />
    </root>
</configuration>

```

上面配置的含义是:

- 所有日志 → 按指定格式 → 同时输出到控制台和文件
- 文件按“时间+大小”自动切割, 保留30天

日志级别:

从低到高 (从“最详细”到“最严重”) :

```
TRACE < DEBUG < INFO < WARN < ERROR
```

- `trace`: 追踪、记录程序运行的所有信息(使用较少)
- `debug`: 调试、记录程序运行过程中出现的信息, 实际应用中一般将其视为最低级别(使用较多)
- `info`: 记录一般信息, 描述程序运行的关键事件, 如: 网络连接、io操作(使用较多)
- `warn`: 警告信息, 记录潜在有害的情况(使用较多)
- `error`: 错误信息(使用较多)
- 根据配置的日志级别, 输出此级别及以上级别的日志信息

创建日志器的简易方法:

- 在类中添加 `@Slf4j` 注解

```
@Slf4j
@RequestMapping("/depts")
@RestController
public class DepController {
    @Autowired
    private DeptService deptService;
    // @RequestMapping(value = "/depts", method = RequestMethod.GET)
    @GetMapping
    public Result list(){
        // System.out.println("查询全部部门数据");
        log.info("查询全部部门数据");
        List<Dept> deptList = deptService.findAll();
        return Result.success(deptList);
    }
    // 省略...
}
```

员工管理(功能开发):

多表关系:

- 一对多
- 一对一
- 多对多

一对多:

- 一个部门对应多个员工

外键约束：确保数据表之间的完整性和一致性

```
alter table emp add constraint fk_emp_dept_id foreign key (dept_id) references dept(id);
```

给 emp 表添加一个外键约束：emp.dept_id 必须引用 dept 表中的 id

两种外键约束：

- 物理外键:使用 foreign key 定义外键
- 逻辑外键(推荐):在业务逻辑中,解决外键关联

一对一：

- 用户与身份证信息的关系

多对多：

- 学生与课程的关系
- 关系:一个学生可以选修多门课程,一门课程也可以供多个学生选择
- 实现:建立第三张中间表,中间表至少包含两个外键,分别关联两方主键

多表设计案例：

```
-- 多表设计：案例
-- 表：dep(1) -----> emp(n)  emp(1) -----> emp_expr(n)
create table emp_expr(
    id int unsigned primary key auto_increment comment 'ID,主键',
    begin date comment '开始时间',
    end date comment '结束时间',
    company varchar(50) comment '公司名称',
    job varchar(50) comment '职位',
    emp_id int unsigned comment '关联的员工ID'
) comment '工作经历表'
```

多表查询：

- 内连接:两表交集
- 左外连接:两表交集+左表所有数据
- 右外连接:两表交集+右表所有数据
- 子查询:查询的嵌套(在查询的结果中再查询一次)

多表查询SQL语句：

```
-- 查询
select * from emp,dept where emp.dept_id = dept.id;
```

在多表查询时,需要消除无效的笛卡尔积

内连接：

```
-- ===== 内连接 =====
-- A. 查询所有员工的ID, 姓名, 及所属的部门名称 (隐式、显式内连接实现)
```

```

-- 隐式
select emp.id , emp.name , dept.name from emp,dept where emp.dept_id = dept.id;

-- 显式
select emp.id , emp.name , dept.name from emp inner join dept on emp.dept_id = dept.id;

select emp.id , emp.name , dept.name from emp join dept on emp.dept_id = dept.id;

-- B. 查询 性别为男, 且工资 高于8000 的员工的ID, 姓名, 及所属的部门名称 (隐式、显式内连接实现)
-- 隐式
select emp.id , emp.name , dept.name from emp,dept where emp.dept_id = dept.id and
emp.gender = 1 and emp.salary > 8000;

-- 显式
select emp.id , emp.name , dept.name from emp join dept on emp.dept_id = dept.id where
emp.gender = 1 and emp.salary > 8000;

-- 为表起别名
select e.id , e.name , d.name from emp e join dept d on e.dept_id = d.id where e.gender = 1
and e.salary > 8000;

```

外连接:

```

-- ===== 外连接 =====
-- A. 查询员工表 所有 员工的姓名, 和对应的部门名称 (左外连接)
select e.name,d.name from emp e left join dept d on e.dept_id = d.id;

-- B. 查询部门表 所有 部门的名称, 和对应的员工名称 (右外连接)
select d.name,e.name from emp e right join dept d on e.dept_id = d.id;

-- C. 查询工资 高于8000 的 所有员工的姓名, 和对应的部门名称 (左外连接)
select e.name,d.name from emp e left join dept d on e.dept_id = d.id where e.salary > 8000;

-- 右外连接
select e.name,d.name from dept d right join emp e on e.dept_id = d.id where e.salary >
8000;

```

子查询(查询的嵌套):

- **标量子查询:**子查询返回的结果为单个值
- **列子查询:**子查询返回的结果为一列
- **行子查询:**子查询返回的结果为一行
- **表子查询:**子查询返回的结果为多行多列

```

-- ===== 子查询 =====
-- 标量子查询
-- A. 查询 最早入职 的员工信息
-- a. 获取最早入职时间
select min(entry_date) from emp;

```

```

-- b.查询 最早入职 的员工信息
select * from emp where entry_date = '2000-01-01';

select * from emp where entry_date = (select min(entry_date) from emp);

-- B. 查询在 "阮小五" 入职之后入职的员工信息
-- a. 查询"阮小五"的入职时间
select entry_date from emp where name = '阮小五';

-- b. 查询在 该时间 之后入职的员工信息
select * from emp where entry_date > '2015-01-01' ;

select * from emp where entry_date > (select entry_date from emp where name = '阮小五');


-- 列子查询
-- A. 查询 "教研部" 和 "咨询部" 的所有员工信息
-- a. 查询 "教研部" 和 "咨询部" 的部门ID
select id from dept where name = '教研部' or name = '咨询部';

-- b. 查询指定部门ID的员工信息
select * from emp where dept_id in (2,3);

select * from emp where dept_id in (select id from dept where name = '教研部' or name = '咨询部');


-- 行子查询
-- A. 查询与 "李忠" 的薪资 及 职位都相同的员工信息 ;
-- a. 查询与 "李忠" 的薪资 及 职位
select salary,job from emp where name = '李忠';

-- b. 查询指定薪资和职位的员工信息
select * from emp where salary = 5000 and job = 5;

select * from emp where salary = (select salary from emp where name = '李忠') and job =
(select job from emp where name = '李忠');

-- 优化:
select * from emp where (salary,job) = (5000,5);

select * from emp where (salary,job) = (select salary,job from emp where name = '李忠');


-- 表子查询
-- A. 获取每个部门中薪资最高的员工信息
-- a. 获取每个部门的最高薪资
select dept_id,max(salary) from emp group by dept_id;

```

```
-- b. 查询每个部门中薪资最高的员工信息
select * from emp e, (select dept_id,max(salary) max_sal from emp group by dept_id) a
  where e.dept_id = a.dept_id and e.salary = a.max_sal;
```

表子查询:

```
-- 表子查询
-- A. 获取每个部门中薪资最高的员工信息
-- a. 获取每个部门的最高薪资
select dept_id,max(salary) from emp group by dept_id;

-- b. 查询每个部门中薪资最高的员工信息
select * from emp e, (select dept_id,max(salary) max_sal from emp group by dept_id) a
  where e.dept_id = a.dept_id and e.salary = a.max_sal;
```

其中 a. 的sql语句可以看做一张表,然后拿到 b. 里面去查询

多表查询案例:

```
-- 需求:
-- 1. 查询 "教研部" 性别为 男, 且在 "2011-05-01" 之后入职的员工信息 。
-- 自己写的:
select id from dept where name = '教研部';
select * from emp where entry_date > '2011-05-01';
select * from emp where dept_id = (select id from dept where name = '教研部') and entry_date > '2011-05-01';
-- 自己写的是子查询, 答案写的是多表连接一般情况下性能更好适合复杂查询
-- 答案:
select e.* from emp e,dept d where e.dept_id = d.id and d.name = '教研部' and e.gender = 1
and e.entry_date > '2011-05-01';

-- 2. 查询工资 低于公司平均工资的 且 性别为男 的员工信息 。
-- a. 计算平均薪资
select avg(salary) from emp;
-- b. 查询低于 平均薪资 且 性别为男 的员工信息
select * from emp where salary < (select avg(salary) from emp) and gender = 1;

-- 3. 查询部门人数超过 10 人的部门名称 。
-- 表: dept,emp
select d.name,count(*) from emp e , dept d where e.dept_id = d.id group by d.name having
count(*) > 10;

-- 4. 查询在 "2010-05-01" 后入职, 且薪资高于 10000 的 "教研部" 员工信息, 并根据薪资倒序排序。
-- 表: dept,emp
select * from emp e, dept d where e.dept_id =d.id and e.entry_date > '2010-05-01'
and e.salary > 10000 and d.name = '教研部' order by e.salary
desc ;

-- 5. 查询工资 低于本部门平均工资的员工信息 。
-- 5.1 查询每个部门的平均薪资
```

```
select dept_id,avg(salary) avg_sal from emp group by dept_id;
```

-- 5.2 查询低于平均薪资的员工信息

```
select e.* from emp e,(select dept_id,avg(salary) avg_sal from emp group by dept_id) a
    where e.dept_id = a.dept_id and e.salary < a.avg_sal;
```

员工管理(功能开发):

准备工作:

- 创建相应的 Controller、Service 和 Mapper

分页查询(代码实现):

EmpController.java

```
@Autowired
private EmpService empService;

/**
 * 分页查询
 */
@GetMapping
public Result page(@RequestParam(defaultValue = "1") Integer page,
                   @RequestParam(defaultValue = "10") Integer pageSize){
    log.info("分页查询:{}",page,pageSize);
    PageResult<Emp> pageResult = empService.page(page,pageSize);
    return Result.success(pageResult);
}
```

EmpServiceImpl.java

```
@Autowired
private EmpMapper empMapper;
@Override
public PageResult<Emp> page(Integer page, Integer pageSize) {
    // 1. 调用mapper接口,查询总记录数
    Long total = empMapper.count();

    // 2. 调用mapper接口,查询结果列表
    //page: 当前第几页 (从1开始)
    //pageSize: 每页多少条数据
    Integer start = (page - 1) * pageSize;
    List<Emp> rows = empMapper.list(start, pageSize);

    // 3. 封装结果 PageResult
    return new PageResult<>(total, rows);
}
```

EmpMapper.java

```
/**
 * 员工信息
```

```

    */
    @Mapper
    public interface EmpMapper {

        /**
         * 查询总记录数
         */
        @Select("select count(*) from emp e left join dept d on e.dept_id = d.id")
        public Long count();

        /**
         * 分页查询
         */
        @Select("select e.*,d.name deptName from emp e left join dept d on e.dept_id = d.id " +
            "order by e.update_time desc limit #{start},#{pageSize}")
        public List<Emp> list(Integer start,Integer pageSize);
    }

```

Apifox测试(GET):

```
http://localhost:8080/emps?page=1&pageSize=5
```

分页查询(PageHelper分页插件):

PageHelper:

`PageHelper`是第三方提供的在Mybatis框架中用来实现分页的插件,用来简化分页操作,提高开发效率。

pom.xml

```

<!--分页插件PageHelper-->
<dependency>
    <groupId>com.github.pagehelper</groupId>
    <artifactId>pagehelper-spring-boot-starter</artifactId>
    <version>1.4.7</version>
</dependency>

```

EmpMapper.java

```

    /**
     * 分页查询
     */
    @Select("select e.*,d.name deptName from emp e left join dept d on e.dept_id = d.id
        order by e.update_time desc")
    public List<Emp> list();

```

EmpServiceImpl.java

```

    /**
     * PageHelper分页查询
     * @param page 页码
     * @param pageSize 每页记录数

```



```

    */
    @Override
    public PageResult<Emp> page(Integer page, Integer pageSize) {
        // 1. 设置分页参数(PageHelper)
        PageHelper.startPage(page, pageSize);

        // 2. 执行查询
        List<Emp> empList = empMapper.list();

        // 3. 解析查询结果,并封装数据
        Page<Emp> p = (Page<Emp>) empList;

        return new PageResult<Emp>(p.getTotal(), p.getResult());
    }

```

PageHelper实现机制:

- PageHelper 定义的SQL语句不能加分号
- PageHelper 仅仅能对紧跟在其后的第一个查询语句进行分页处理

分页查询(条件分页查询):